SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

A PROJECT REPORT ON

TrustDeploy

Container Image Supply Chain Security Using Digital Signatures

Submitted in partial fulfilment of the requirements of
ACC

Submitted by

Name of the Students	Roll No.
Archit Singh	16010123057
Anish Patni	16010123045
Amrit Nigam	16010123038
Rishi Shanbhag	16010123256

Under the guidance of

Prof. Sheetal Pereira

Department of Computer Engineering
K. J. Somaiya School of Engineering
Somaiya Vidyavihar University

Academic Year 2023-27

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

CERTIFICATE

This is to certify that the project entitled **"TrustDeploy: Container Image Supply Chain Security Using Digital Signatures"** is a bonafide work carried out by the students Archit Singh (Roll No. 16010123057), Anish Patni (Roll No. 16010123045), Amrit Nigam (Roll No. 16010123038) and Rishi Shanbhag (Roll No. 16010123256) in partial fulfilment of the requirements of ACC in the Department of Computer Engineering, K. J. Somaiya School of Engineering, Somaiya Vidyavihar University, during the academic year 2023-27, under my guidance.

Prof. Sheetal Pereira	
Project Guide	Head of Department

Date: 21/10/2026

Place: Mumbai

ABSTRACT

Modern software is shipped as container images that pass through build servers, registries and deployment pipelines before they run in production. Each hand-off is an opportunity for an attacker to substitute, modify or re-label an image, and most pipelines deploy whatever a mutable tag such as `:1.0.0` happens to point to at that moment. This project, **TrustDeploy**, implements a local, zero-cost prototype of a container image supply chain that only deploys images which carry a valid digital signature from a trusted release key.

A sample Python web service is built with Docker and pushed to a local OCI registry (`registry:2` on `localhost:5001`). The approved image is signed **by its immutable SHA-256 digest** with Sigstore Cosign (v3.1.3) using an encrypted ECDSA P-256 key pair whose private half never enters the repository or the image. A verification engine written in standard-library Python resolves every requested tag to its digest, verifies the Cosign signature against the trusted public key, checks that the signed payload names exactly that digest, and hands the result to a fail-closed policy gate. Every decision, accepted or rejected, is appended to a JSON-lines audit log that can be exported as HTML, CSV and JSON. The same trust policy is also enforced at Kubernetes admission time on a local kind cluster using the Sigstore Policy Controller.

Ten end-to-end test cases (TC01 to TC07, with sub-cases) and eight unit tests were executed. The signed image was accepted, while unsigned images, images signed with an attacker's key, a genuine signature copied onto another image, a trusted tag overwritten with a tampered build, a missing verification tool, an unreachable registry and an image from a non-approved registry were all rejected with a specific reason code. The recorded evaluation run produced 15 audit events (4 accepted, 11 rejected) and every outcome matched its expected result. The prototype shows that signature-based admission control is practical to adopt with open-source tools and no cloud spend, while also documenting its limits: a signature proves origin and integrity, not the absence of vulnerabilities.

Keywords: software supply chain security, container images, digital signatures, Sigstore, Cosign, OCI digest, admission control, policy as code, audit logging, Kubernetes.

TABLE OF CONTENTS

Abstract ..... **iii**
List of Figures ..... **vi**
List of Tables ..... **vii**
Chapter 1: Introduction **1**
1.1 Background 1
1.2 Supply-Chain Threats to Container Images 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope 3
1.6 Organisation of the Report 3
Chapter 2: Literature Review and Background **4**
2.1 OCI Images, Tags and Digests 4
2.2 Digital Signatures for Software Artefacts 4
2.3 Sigstore and Cosign 4
2.4 Signature Storage: Legacy Tags, Bundles and OCI Referrers 5
2.5 Transparency Logs 5
2.6 Admission Control and the Sigstore Policy Controller 5
2.7 Summary and Research Gap 6
Chapter 3: System Architecture and Design **7**
3.1 Architectural Overview 7
3.2 Components 7
3.3 Trust Model 8
3.4 Verification and Decision Flow 8
3.5 Reason Codes 9
3.6 Audit Record Design 10
3.7 Key Design Decisions 10
Chapter 4: Technology Stack **11**
Chapter 5: Implementation Methodology **13**
5.1 Phase 1: Environment Setup 13
5.2 Phase 2: Building the Sample Application 13
5.3 Phase 3: Key Generation and Signing 14
5.4 Phase 4: Verification Engine 16
5.5 Phase 5: Attack Simulation and Untrusted Images 18
5.6 Phase 6: Policy Enforcement in the Deployment Workflow 19
5.7 Phase 7: Audit Logging and Reporting 22
Chapter 6: Testing **23**
6.1 Test Strategy 23
6.2 Test Case Summary 23

6.3 Test Case Details....24
6.4 Unit Tests....30
Chapter 7: Results and Discussion....31
7.1 Summary of Results....31
7.2 Effectiveness Against the Threat Model....31
7.3 Performance....32
7.4 Discussion....32
Chapter 8: Security Considerations and Limitations....33
Chapter 9: Cost Analysis....34
Chapter 10: Future Enhancements....35
Chapter 11: Conclusion....36
References....37
Appendix A: Reproducing the Demonstration....38

LIST OF FIGURES

Figure 3.1: TrustDeploy system architecture....7
Figure 3.2: Verification and decision flow of the policy gate....9
Figure 4.1: Environment: tool versions, local registry and project layout....12
Figure 5.1: Image built, pushed and its immutable digest recorded; smoke test of the service....14
Figure 5.2: Release key pair generated; private key kept outside the repository and image....15
Figure 5.3: Approved image signed by digest; cosign tree shows the .sig tag and the OCI referrer bundle....16
Figure 5.4: TC01: signed image verified with Cosign and accepted by the TrustDeploy gate....18
Figure 5.5: Preparation of the unsigned, attacker-signed, forged and tampered images....19
Figure 5.6: Deployment workflow: attacker image blocked (exit 2), approved image deployed by digest....20
Figure 5.7: kind cluster, Policy Controller release and webhook pod, and the ClusterImagePolicy 21
Figure 5.8: Kubernetes admission: signed image admitted; attacker-signed and unsigned images denied....22
Figure 6.1: TC02: unsigned image rejected (NO_SIGNATURE)....24
Figure 6.2: TC03: image signed with an attacker key rejected (UNTRUSTED_SIGNER)....25
Figure 6.3: TC04a: copied genuine signature rejected (SIGNATURE_IMAGE_MISMATCH). .25
Figure 6.4: TC04b: trusted tag pointed at a tampered build, rejected (NO_SIGNATURE)....26
Figure 6.5: TC05a: Cosign missing, deployment of a valid image blocked (fail closed)....26
Figure 6.6: TC05b: registry unreachable, deployment blocked (IMAGE_RESOLUTION_FAILED)....27
Figure 6.7: TC05c: image from a non-approved registry rejected (REGISTRY_NOT_ALLOWED)....27
Figure 6.8: TC06: four images verified in one batch, four separate audit entries....28
Figure 6.9: TC07: audit report export (HTML, CSV, JSON) with the full audit trail....29
Figure 6.10: TC07: generated HTML audit dashboard....29
Figure 6.11: Unit tests: 8 of 8 passed....30

LIST OF TABLES

Table 1.1: Supply-chain threats addressed by TrustDeploy 1
Table 3.1: System components and their implementation 7
Table 3.2: Reason codes produced by the verification engine 9
Table 3.3: Fields of an audit record 10
Table 4.1: Technology stack 11
Table 6.1: End-to-end test cases and observed outcomes 23
Table 6.2: Unit tests of the fail-closed gate 30
Table 7.1: Distribution of reason codes in the evaluation run 31
Table 7.2: Threat coverage 31
Table 9.1: Cost of the prototype compared with a managed cloud registry 34

CHAPTER 1
INTRODUCTION

1.1 Background

Containers have become the standard unit of deployment for cloud-native applications. A container image bundles an application together with its runtime, libraries and configuration, and is distributed through an *OCI registry*. The deployment platform (a Docker host, a Kubernetes cluster or a managed service) pulls the image from the registry and runs it. Because the same image moves unchanged from build to production, it is attractive to attackers: compromising one image, or one step that handles it, can compromise every system that runs it.

The path from source code to a running container is called the *software supply chain*. For container images it typically includes the source repository, the build system, the base images it pulls, the registry that stores the result and the pipeline that deploys it. A deployment platform that simply pulls `name:tag` trusts every one of those components implicitly. TrustDeploy replaces that implicit trust with an explicit, cryptographically checked one: an image is deployed only if it was signed by the organisation's release key.

1.2 Supply-Chain Threats to Container Images

The threats that motivated the project are summarised in Table 1.1. Each of them is later reproduced as a test case against the prototype.

Table 1.1: Supply-chain threats addressed by TrustDeploy

Threat	How it happens	Test case
Unsigned / unapproved image	An image is pushed to the registry without going through the release process (for example, a developer shortcut or a compromised CI job).	TC02
Image signed by an untrusted party	An attacker builds a malicious image and signs it with their own key so that it looks "signed".	TC03
Signature replay onto another image	An attacker who cannot reach the private key copies a genuine signature from an approved image and attaches it to their own image.	TC04a
Tag hijacking	Tags are mutable. An attacker with push access re-points the trusted tag `:1.0.0` to a tampered build; anything that deploys by tag silently runs the new image.	TC04b
Verification infrastructure failure	The verifier is missing, crashes, times out or cannot reach the registry. A naive gate may fail open and deploy anyway.	TC05a, TC05b

Threat	How it happens	Test case
Untrusted registry	An image is pulled from a public or unknown registry outside organisational control.	TC05c
Lack of accountability	Without a record of what was checked and why it was allowed or blocked, incidents cannot be investigated.	TC06, TC07

1.3 Problem Statement

Container deployment pipelines commonly trust an image because of *where it is* (a registry) and *what it is called* (a tag), not because of *who produced it* or *whether it is the exact artefact that was approved*. Tags are mutable, registries can be written to by anyone with credentials, and verification steps, when present, often fail open when something goes wrong. The problem addressed by this project is therefore:

> *To design and implement a deployment gate that allows a container image to be deployed only when it is the exact, digest-identified artefact that was signed by a trusted release key, that blocks every other case including errors in the verification process itself, and that records an auditable reason for every decision.*

1.4 Objectives

1. Build and publish a sample containerised application to a local OCI registry and record its immutable digest.
2. Generate a protected signing key pair and sign the approved image by digest using Sigstore Cosign.
3. Implement a verification engine that resolves tags to digests and verifies signatures against a trusted public key.
4. Implement a fail-closed policy gate that accepts only an explicit, successful verification.
5. Enforce the gate inside a deployment workflow so that rejected images are never started.
6. Reproduce realistic attacks (unsigned image, attacker key, copied signature, tag hijack) and failure conditions (missing tool, registry outage, non-approved registry) and show that each is blocked.
7. Record every decision in a structured audit log and export it as HTML, CSV and JSON reports.
8. Extend enforcement to Kubernetes admission control with the Sigstore Policy Controller.

9. Keep the entire prototype local-first with zero cloud cost.

1.5 Scope

In scope: key-based signing and verification of OCI container images with Cosign; digest pinning; a policy file with trusted keys, allowed registries and a timeout; deployment to a local Docker host and to a local kind Kubernetes cluster; audit logging and reporting; automated end-to-end and unit tests.

Out of scope: vulnerability scanning of image contents, SBOM and provenance attestations (builds use `--provenance=false --sbom=false`), keyless (OIDC) signing with the public Sigstore infrastructure, transparency-log upload, hardware-backed key storage, and production-grade high availability. These are discussed as limitations and future enhancements.

1.6 Organisation of the Report

Chapter 2 reviews the background technologies. Chapter 3 presents the system architecture and design decisions. Chapter 4 lists the technology stack. Chapter 5 describes the implementation phase by phase with code excerpts. Chapter 6 documents the test cases and their evidence, and Chapter 7 discusses the results. Chapter 8 covers security considerations and limitations, Chapter 9 the cost analysis, Chapter 10 future enhancements and Chapter 11 concludes the report.

CHAPTER 2

LITERATURE REVIEW AND BACKGROUND

2.1 OCI Images, Tags and Digests

An OCI image is described by a *manifest*, a JSON document that lists the image configuration and its filesystem layers by content hash. The SHA-256 hash of the manifest is the image *digest*, written `sha256:<64 hex>`. Because it is derived from the content, a digest is immutable: changing a single byte of any layer changes the layer hash, the manifest and therefore the digest. A reference of the form `repo@sha256:...` always denotes exactly one image.

A *tag* such as `sample-app:1.0.0` is only a human-friendly pointer stored in the registry. It can be moved to a different manifest at any time by anyone with push access. Deploying by tag therefore means deploying "whatever the tag points to right now", which is the root cause of the tag-hijacking attack (TC04b). Signing and deploying by digest removes this ambiguity, and TrustDeploy always resolves a tag to a digest before verifying and deploys the verified digest rather than the tag.

2.2 Digital Signatures for Software Artefacts

A digital signature scheme uses a private key to produce a signature over a message and the corresponding public key to check it. A valid signature gives two guarantees: *integrity* (the message has not changed since it was signed) and *authenticity* (it was signed by the holder of the private key). TrustDeploy uses ECDSA over the NIST P-256 curve, the default key type produced by `cosign generate-key-pair`. The signed message is not the image itself but a small JSON payload that names the image digest; since the digest commits to every byte of the image, signing the payload is equivalent to signing the image.

2.3 Sigstore and Cosign

Sigstore is an open-source project, hosted by the OpenSSF, that provides tooling for signing and verifying software artefacts. Its command-line tool *Cosign* signs container images and stores the signatures in the same OCI registry as the image, so no separate signature server is needed [1]. Cosign supports two modes:

- **Key-based signing**, where the user generates a key pair (`cosign generate-key-pair`), signs with `cosign sign --key cosign.key` and verifies with `cosign verify --key cosign.pub` [1], [2]. This mode works fully offline and is used by TrustDeploy.

- **Keyless signing**, where a short-lived certificate is issued by the Fulcio certificate authority against an OIDC identity and the signature is recorded in the Rekor transparency log. Verification then checks the certificate identity and issuer rather than a static key [2].

`cosign verify` checks that a signature exists for the given image, that it validates against the supplied key or identity, and that the signed claims refer to the image being verified [2]. On success it prints the verified payloads as JSON, which TrustDeploy parses for an additional digest check.

2.4 Signature Storage: Legacy Tags, Bundles and OCI Referrers

Historically Cosign stored a signature as a separate image whose tag is derived from the signed digest, `sha256-<digest>.sig`. Newer releases store a *Sigstore bundle* (signature, verification material and the signed statement in one document) and attach it to the image through the OCI *referrers* mechanism, where the signature manifest carries a `subject` field pointing at the image digest. Registries that do not implement the referrers API, such as the local `registry:2`, are served through the referrers *tag-schema fallback*: an index tagged `sha256-<digest>` lists the referring manifests. Both artefacts are visible in Figure 5.3, and the fallback index is exactly what the attack in TC04a manipulates.

2.5 Transparency Logs

A transparency log is an append-only, cryptographically verifiable ledger. Sigstore's Rekor log records signing events so that anyone can audit which signatures were produced and when, and so that a signature can be shown to have existed while a short-lived certificate was valid. By default Cosign uploads to and verifies against Rekor. Because this prototype is designed to run offline and at zero cost, signing uses `--tlog-upload=false` and verification uses `--insecure-ignore-tlog=true`. The trust decision then rests entirely on the protected key, which is acceptable for a closed prototype but is listed as a limitation in Chapter 8.

2.6 Admission Control and the Sigstore Policy Controller

Kubernetes *admission controllers* intercept requests to the API server after authentication and before objects are persisted. A *validating admission webhook* can reject a Pod whose image does not meet a policy. The Sigstore Policy Controller is such a webhook: it evaluates `ClusterImagePolicy` resources that match image globs to authorities (keys or keyless identities) and denies Pods whose images are not signed accordingly [3]. It is installed with

Helm into the `cosign-system` namespace and is enforced only in namespaces carrying the label `policy.sigstore.dev/include=true` [4]. It also resolves tags to digests at admission time, so the running Pod is pinned to the verified image.

2.7 Summary and Research Gap

The building blocks for signed container images exist and are mature, but a typical student or small-team deployment still pulls images by tag without verification. Documentation tends to show a single successful `cosign verify`, not how a pipeline should behave when verification fails, when the tool or registry is unavailable, or when an attacker re-uses a genuine signature. TrustDeploy addresses this gap with an end-to-end, reproducible prototype that focuses on the *negative* paths: it demonstrates that each attack and failure mode is blocked, explains why, and leaves an audit trail.

CHAPTER 3
SYSTEM ARCHITECTURE AND DESIGN

3.1 Architectural Overview

TrustDeploy separates the *trusted build and release* side, where images are produced and signed, from the *deployment* side, where images are verified before they are allowed to run (Figure 3.1). The only things the two sides share are the registry and the public half of the release key. An attacker is assumed to be able to write to the registry (push images, push signatures, move tags) but not to read the private key.

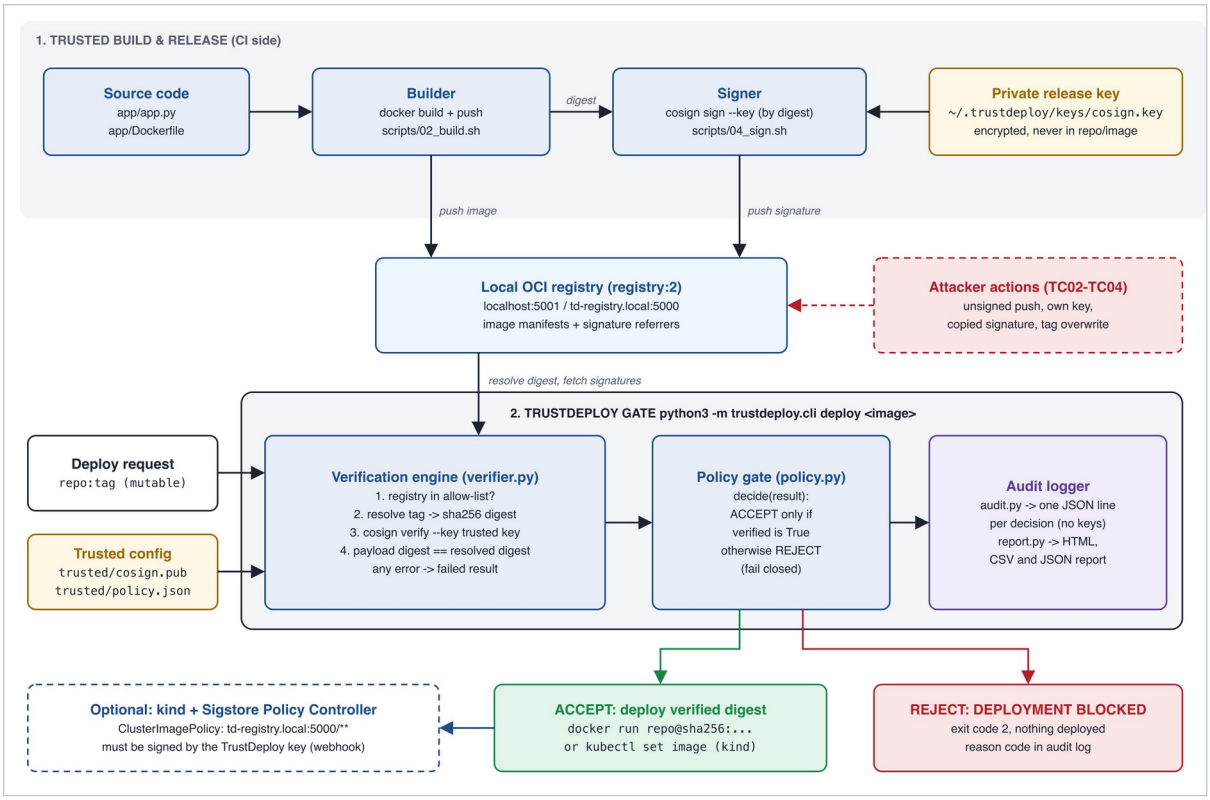


Figure 3.1: TrustDeploy system architecture

3.2 Components

Table 3.1: System components and their implementation

Component	Responsibility	Implementation
Builder	Builds the sample app image, pushes it to the registry and records the immutable digest of the approved release.	`scripts/02_build.sh`, `app/Dockerfile`
Signer	Signs the approved image by digest with the private release key in two formats (bundle via OCI referrer, legacy `.sig`).	`scripts/04_sign.sh` (Cosign v3.1.3)
Trusted key and policy	Public key, allowed registries and timeout that define what "trusted" means.	`trusted/cosign.pub`, `trusted/policy.json`

Component	Responsibility	Implementation
Verification engine	Allow-list check, tag-to-digest resolution, `cosign verify`, payload digest check, classification of failures.	`trustdeploy/verifier.py`
Policy gate	Converts a verification result into ACCEPT or REJECT, failing closed.	`trustdeploy/policy.py`
Deployment workflow	`verify`, `deploy` and `report` commands; deploys only the verified digest.	`trustdeploy/cli.py`
Audit logger and reports	Appends one JSON line per decision; exports HTML, CSV and JSON.	`trustdeploy/audit.py`, `trustdeploy/report.py`
Kubernetes admission (optional layer)	Denies unsigned or wrongly signed Pods at the API server.	kind + Sigstore Policy Controller, `k8s/*.yaml`

3.3 Trust Model

- **Root of trust:** the ECDSA P-256 release key. The private key is encrypted with a random password and stored in `~/.trustdeploy/keys` (directory mode 700, files mode 600), outside the project and outside every image. `.gitignore` excludes `*.key`.
- **Trusted configuration:** `trusted/cosign.pub` and `trusted/policy.json` are version-controlled; changing them is a reviewable act. The audit log records the SHA-256 fingerprint of each trusted key in use.
- **Untrusted inputs:** image names, tags, registry contents and the output of external tools. None of these can cause an ACCEPT without a successful cryptographic check.
- **Fail-closed principle:** any error, timeout, missing tool or unexpected output results in REJECT.

3.4 Verification and Decision Flow

Figure 3.2 shows the order of checks performed for every request. Cheap, local checks come first: an image from a registry outside the allow-list is rejected without any network access. The tag is then resolved to a digest so that the signature is checked, and the deployment later performed, against one immutable image.

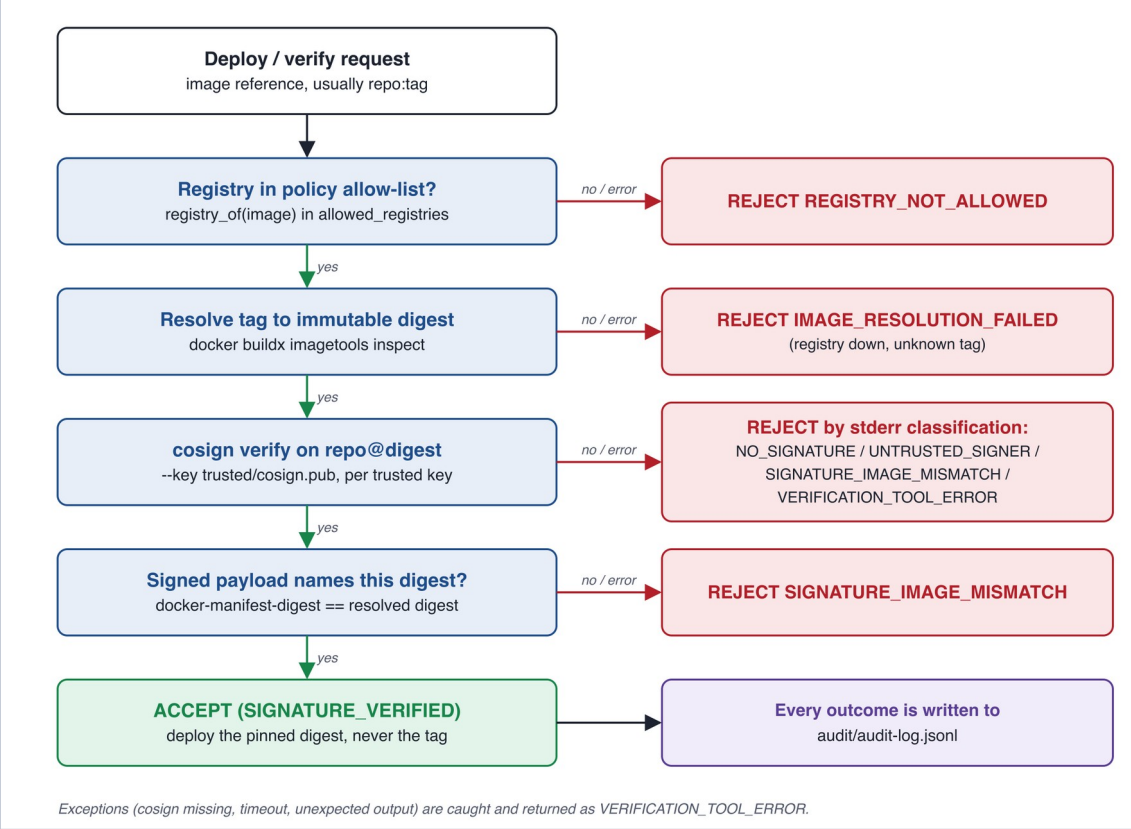


Figure 3.2: Verification and decision flow of the policy gate

3.5 Reason Codes

Each decision carries a machine-readable reason code (Table 3.2). Distinguishing the reasons matters operationally: `NO_SIGNATURE` usually indicates a process gap, `UNTRUSTED_SIGNER` or `SIGNATURE_IMAGE_MISMATCH` indicate an attack, and `IMAGE_RESOLUTION_FAILED` or `VERIFICATION_TOOL_ERROR` indicate an infrastructure fault.

Table 3.2: Reason codes produced by the verification engine

Reason code	Decision	Meaning
`SIGNATURE_VERIFIED`	ACCEPT	A signature from a trusted key validates and its payload names the resolved digest.
`NO_SIGNATURE`	REJECT	The image has no Cosign signature in the registry.
`UNTRUSTED_SIGNER`	REJECT	Signatures exist, but none validate against a trusted public key.
`SIGNATURE_IMAGE_MISMATCH`	REJECT	A signature exists but was issued for a different image digest.
`IMAGE_RESOLUTION_FAILED`	REJECT	The tag could not be resolved to a digest (registry down, unknown tag).
`VERIFICATION_TOOL_ERROR`	REJECT	Cosign missing, timed out, or produced unexpected output.
`REGISTRY_NOT_ALLOWED`	REJECT	The image's registry is not in the policy allow-list.

3.6 Audit Record Design

The audit log is an append-only JSON-lines file, `audit/audit-log.jsonl`. JSON lines were chosen because each record is independent, the file can be appended to safely by one process, and it is trivially ingested by log tooling. The record never contains key material, only a fingerprint of the trusted public key.

Table 3.3: Fields of an audit record

Field	Description
`event_id, timestamp_utc`	Random UUID and UTC timestamp of the decision.
`action, test_label`	`verify` or `deploy`, and an optional label such as TC03.
`image_ref, resolved_digest, verified_ref`	What was requested, the digest it resolved to, and the digest-pinned reference that was verified.
`decision, reason_code, reason`	ACCEPT or REJECT, the reason code and a human-readable explanation.
`policy, trusted_key_sha256`	Policy name and the 16-hex-digit SHA-256 fingerprint of each trusted key.
`tool_exit_code, tool_stderr_excerpt`	Cosign's exit code and the last 600 characters of its error output, kept as evidence.
`duration_ms, operator`	Time taken by verification and the local user who ran it.

3.7 Key Design Decisions

- **Verify and deploy by digest.** The gate resolves the tag once, verifies `repo@digest`, and deploys exactly that reference, closing the time-of-check to time-of-use gap that a tag would leave open.
- **Defence in depth on the payload.** Even when Cosign exits successfully, the gate checks that every verified payload's `docker-manifest-digest` equals the resolved digest.
- **Fail closed by construction.** `decide()` returns ACCEPT only when `verified is True`; every exception path in the verifier returns a failed result rather than raising.
- **No third-party Python dependencies.** The gate uses only the Python standard library and shells out to the audited `cosign` and `docker` binaries, minimising the gate's own supply-chain surface.
- **Two enforcement points.** A pipeline gate (CLI) and a cluster admission webhook enforce the same key, so an image that bypasses the pipeline is still stopped at the cluster.

CHAPTER 4

TECHNOLOGY STACK

All components run on a single developer laptop (macOS on Apple silicon, `darwin/arm64`). The versions in Table 4.1 are those reported by the tools in the evaluation run (Figure 4.1 and Figure 5.7).

Table 4.1: Technology stack

Layer	Technology	Version / detail	Purpose
Container runtime	Docker Engine + Buildx	29.8.1	Build, push, inspect and run images
Signing / verification	Sigstore Cosign	v3.1.3	Key generation, signing, verification, `cosign tree`
Gate implementation	Python 3 (standard library only)	3.14.2 on host	Verifier, policy gate, CLI, audit, reports
Sample application	Python HTTP server on `python:3.12-alpine`	1.0.0	Image under protection; runs as non-root `appuser`
Registry	Distribution `registry:2`	`localhost:5001` (in-cluster alias `td-registry.local:5000`)	Stores images and signatures
Kubernetes	kind (Kubernetes in Docker), kubectl	kind v0.33.0, Kubernetes v1.37.0	Local cluster for admission control
Admission control	Sigstore Policy Controller (Helm)	app 0.13.1, chart 0.10.8	Validating webhook enforcing ClusterImagePolicy
Testing	Bash test runner, Python `unittest`	`tests/cases.sh`, `tests/test_gate.py`	End-to-end and unit tests
Evidence tooling	Pillow (screenshots), headless Chrome	`scripts/termshot.py`, `scripts/htmlshot.sh`	Reproducible evidence images (not used by the gate)

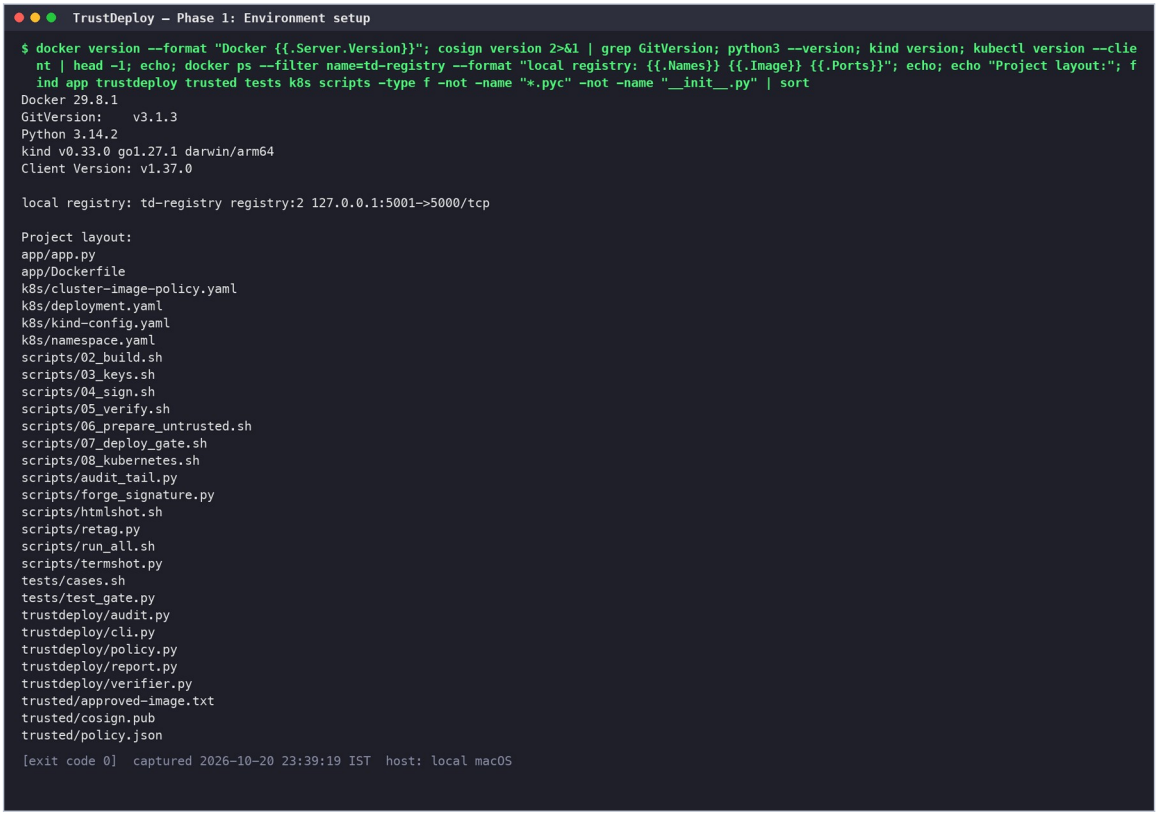


Figure 4.1: Environment: tool versions, local registry and project layout *[01-environment.png]*

The project layout shown in Figure 4.1 separates the gate (`trustdeploy/`), the trusted configuration (`trusted/`), the sample application (`app/`), automation (`scripts/`), Kubernetes manifests (`k8s/`) and tests (`tests/`). `scripts/run_all.sh` reproduces the complete demonstration from a clean registry and audit log.

CHAPTER 5

IMPLEMENTATION METHODOLOGY

The implementation was carried out in seven phases. Each phase is automated by a script so that the whole demonstration can be repeated end to end with `scripts/run_all.sh`, which also captures the evidence figures used in this report.

5.1 Phase 1: Environment Setup

Docker, Cosign, Python, kind and kubectl were installed locally and a private registry was started as a container bound to the loopback interface only:

Listing 5.1: *scripts/run_all.sh (excerpt)*

```
docker run -d --restart=always -p 127.0.0.1:5001:5000 --name td-registry registry:2 >/dev/null
docker network connect --alias td-registry.local kind td-registry 2>/dev/null
```

Connecting the registry container to the `kind` Docker network under the alias `td-registry.local` lets the Kubernetes node pull from the same registry that the host uses on `localhost:5001`.

5.2 Phase 2: Building the Sample Application

The protected workload is a minimal JSON HTTP service. Its Dockerfile uses a small Alpine base and drops root privileges:

Listing 5.2: *app/Dockerfile*

```
FROM python:3.12-alpine
ARG APP_VERSION=1.0.0
ENV APP_VERSION=${APP_VERSION}
RUN adduser -D appuser
WORKDIR /srv
COPY app.py .
USER appuser
EXPOSE 8080
CMD ["python", "app.py"]
```

The build script builds and pushes version 1.0.0, then resolves and records the **digest** of the approved image in `trusted/approved-image.txt`. Provenance and SBOM attestations are disabled so that the pushed reference is a single image manifest whose digest is stable and easy to reason about.

Listing 5.3: *scripts/02_build.sh (excerpt)*

```
docker build -q --provenance=false --sbom=false --build-arg APP_VERSION=1.0.0 -t
    $REG/trustdeploy/sample-app:1.0.0 app >/dev/null
docker push -q $REG/trustdeploy/sample-app:1.0.0 >/dev/null
DIGEST=$(docker buildx imagetools inspect $REG/trustdeploy/sample-app:1.0.0 --format
    '{{json .Manifest}}' | python3 -c 'import json,sys;print(json.load(sys.stdin)["digest"])')
echo "Built and pushed: $REG/trustdeploy/sample-app:1.0.0"
echo "Immutable digest: $DIGEST"
echo "$REG/trustdeploy/sample-app@$DIGEST" > trusted/approved-image.txt
```

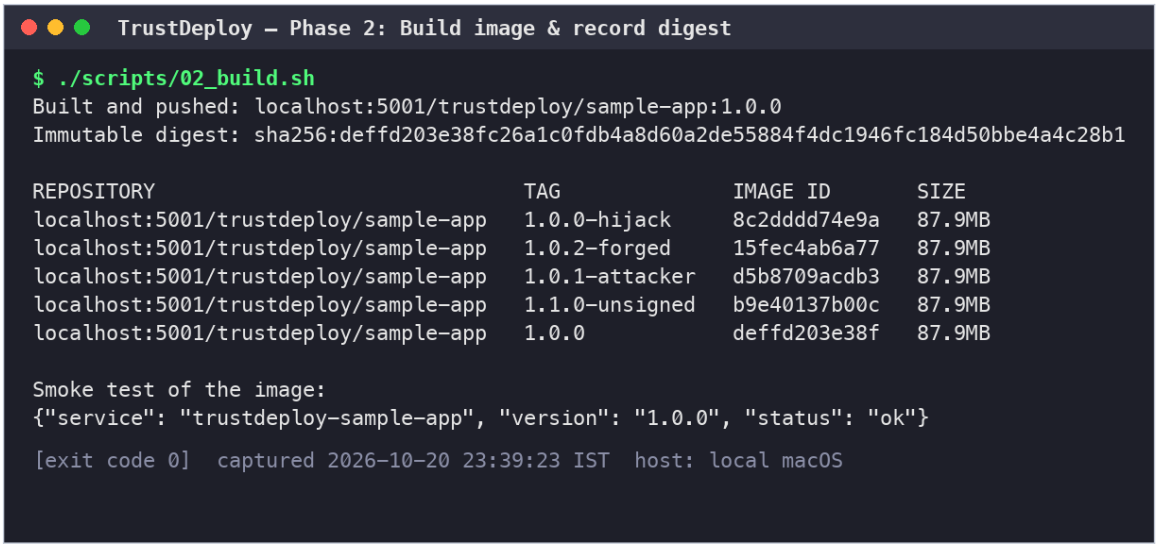


Figure 5.1: Image built, pushed and its immutable digest recorded; smoke test of the service *[02-build-image.png]*

5.3 Phase 3: Key Generation and Signing

A Cosign key pair is generated once. The private key is encrypted with a random 24-byte password produced by OpenSSL, and both live in `~/.trustdeploy/keys`, outside the project. Only the public key is copied into `trusted/cosign.pub`. The script finishes with an explicit check that no `*.key` file exists in the project tree or inside the image (Figure 5.2).

Listing 5.4: *scripts/03_keys.sh (excerpt)*

```
KEYDIR=$HOME/.trustdeploy/keys
mkdir -p "$KEYDIR" && chmod 700 "$KEYDIR"
[ -f "$KEYDIR/password" ] || (openssl rand -base64 24 > "$KEYDIR/password" && chmod 600
    "$KEYDIR/password")
cd "$KEYDIR"
[ -f cosign.key ] || COSIGN_PASSWORD="$(cat password)" cosign generate-key-pair 2>&1
chmod 600 cosign.key
cp cosign.pub "$OLDPWD/trusted/cosign.pub"
```

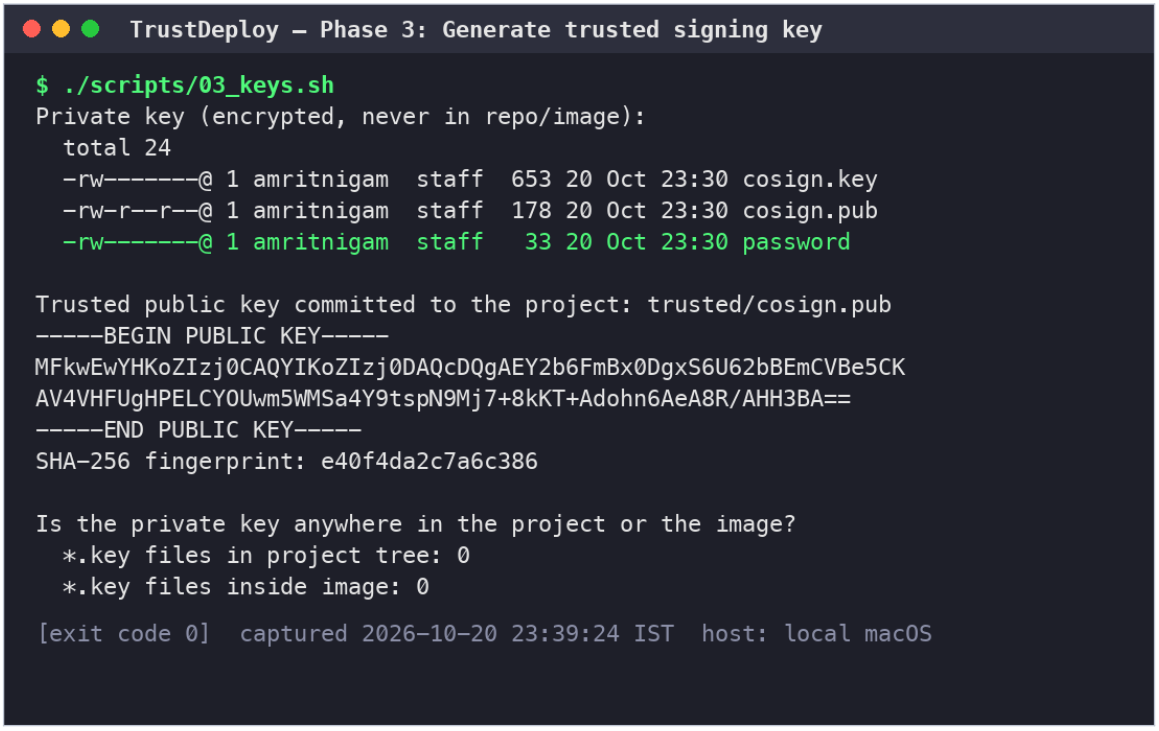


Figure 5.2: Release key pair generated; private key kept outside the repository and image *[03-key-generation.png]*

The approved image is then signed **by digest, never by tag**. It is signed twice with the same key: once in the current Sigstore bundle format attached as an OCI referrer, which the TrustDeploy gate verifies, and once in the legacy `.sig` tag format, which the Kubernetes Policy Controller reads. Transparency-log upload is disabled for offline operation.

Listing 5.5: *scripts/04_sign.sh (excerpt)*

```
REF=$(cat trusted/approved-image.txt)
echo "Approved image (digest-pinned): $REF"; echo
export COSIGN_PASSWORD="$(cat $HOME/.trustdeploy/keys/password)"
echo "-- Sigstore bundle format (OCI referrer), used by the TrustDeploy gate"
cosign sign --key $HOME/.trustdeploy/keys/cosign.key --tlog-upload=false --use-signing-
    config=false -y "$REF" 2>&1 | grep -v deprecated
echo "-- legacy .sig format, read by the Kubernetes Policy Controller admission webhook"
cosign sign --key $HOME/.trustdeploy/keys/cosign.key --tlog-upload=false --use-signing-
    config=false --new-bundle-format=false -y "$REF" 2>&1 | grep -v deprecated
echo; echo "Signature artifacts now attached to the image in the registry:"
cosign tree "$REF" 2>&1
```

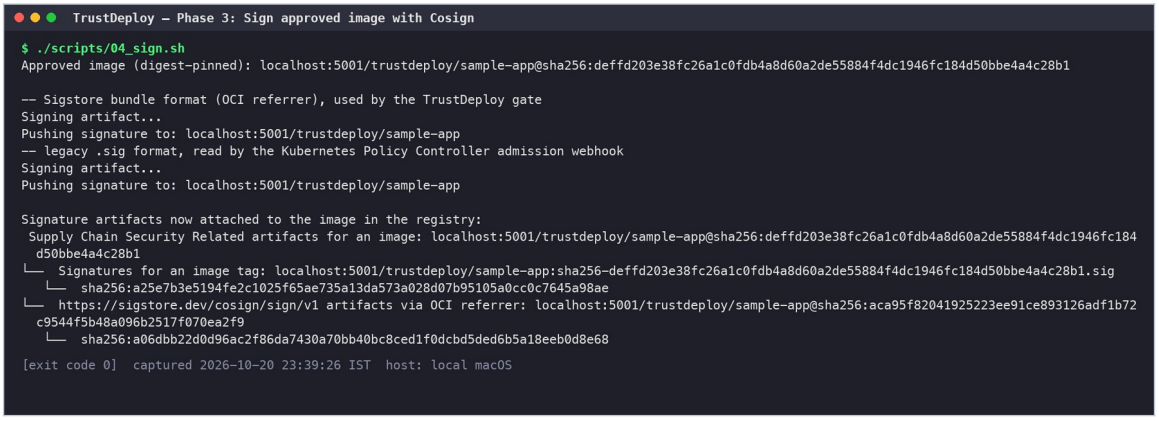


Figure 5.3: Approved image signed by digest; cosign tree shows the .sig tag and the OCI referrer bundle *[04-sign-image.png]*

5.4 Phase 4: Verification Engine

The verification engine in `trustdeploy/verifier.py` is the core of the system. It never raises an exception to its caller: every path returns a `VerificationResult` whose `verified` flag defaults to `False` and whose reason code defaults to `VERIFICATION_TOOL_ERROR`. Tag resolution uses Docker Buildx to read the manifest digest directly from the registry:

Listing 5.6: *trustdeploy/verifier.py: tag-to-digest resolution*

```
def resolve_digest(image_ref, timeout):
    """Resolve a (possibly mutable) tag to the immutable manifest digest in the registry."""
    repo, _, digest = split_ref(image_ref)
    if digest:
        return digest, None
    proc = subprocess.run(
        [DOCKER, "buildx", "imagetools", "inspect", image_ref, "--format", "{{json .Manifest}}"],
        capture_output=True, text=True, timeout=timeout,
    )
    if proc.returncode != 0:
        return None, proc.stderr.strip() or proc.stdout.strip()
    return json.loads(proc.stdout)["digest"], None
```

Signature verification calls `cosign verify` on the digest-pinned reference for each trusted key and then performs an independent check that the verified payload names the resolved digest:

Listing 5.7: *trustdeploy/verifier.py: verify_image()*

```
def verify_image(image_ref, policy):
    start = time.monotonic()
    res = VerificationResult(image_ref=image_ref)
    timeout = policy.get("verification_timeout_seconds", 60)
    try:
        if registry_of(image_ref) not in policy["allowed_registries"]:
            res.reason_code = REGISTRY_NOT_ALLOWED
            res.message = f"Registry '{registry_of(image_ref)}' is not in the policy allow-list."
            return res

        digest, err = resolve_digest(image_ref, timeout)
        if not digest:
            res.reason_code = RESOLVE_ERROR
            res.message = "Could not resolve the image to a digest; failing closed."
            res.tool_stderr = (err or "")[-600:]
            return res
        res.digest = digest
```

```
        res.pinned_ref = f"{split_ref(image_ref)[0]}@{digest}"

        failures = []
        for key in policy["trusted_public_keys"]:
            proc = _cosign_verify(res.pinned_ref, key, timeout)
            res.tool_exit_code = proc.returncode
            if proc.returncode == 0:
                payloads = json.loads(proc.stdout)
                # Defence in depth: every accepted payload must name the exact digest we
    resolved.
                claimed = {p["critical"]["image"]["docker-manifest-digest"] for p in payloads}
                if claimed != {digest}:
                    res.reason_code = DIGEST_MISMATCH
                    res.message = f"Signed payload names {sorted(claimed)}, expected {digest}."
                    return res
                res.verified = True
                res.reason_code = VERIFIED
                res.message = f"Valid signature from trusted key {os.path.relpath(key)} for
    {digest[:19]}…"
                res.signatures = [p.get("optional") or {} for p in payloads]
                return res
            failures.append(proc.stderr.strip())

        res.tool_stderr = "\n".join(failures)[-600:]
        res.reason_code, res.message = classify_failure(res.tool_stderr)
        return res
    except FileNotFoundError as e:
        res.reason_code = TOOL_ERROR
        res.message = f"Verification tool not available ({e.filename}); failing closed."
        return res
    except subprocess.TimeoutExpired:
        res.reason_code = TOOL_ERROR
        res.message = "Verification timed out; failing closed."
        return res
    except Exception as e:  # any unexpected error must still deny
        res.reason_code = TOOL_ERROR
        res.message = f"Unexpected verification error ({type(e).__name__}: {e}); failing closed."
        return res
    finally:
        res.duration_ms = int((time.monotonic() - start) * 1000)
```

When Cosign fails, its error output is classified into a reason code so that the audit log distinguishes an unsigned image from an attack:

Listing 5.8: *trustdeploy/verifier.py: classify_failure()*

```
def classify_failure(stderr):
    s = stderr.lower()
    if "no signatures found" in s or "no matching signatures: no signatures" in s:
        return UNSIGNED, "The image has no Cosign signature in the registry."
    if "does not match any digest in statement" in s or "invalid or missing digest in claim" in
    s:
        return DIGEST_MISMATCH, "A signature exists, but it was issued for a different image
    digest."
    if any(m in s for m in ("no matching signatures", "no matching attestations", "do not match
    threshold", "invalid signature")):
        return UNTRUSTED_KEY, "Signature(s) exist, but none validate against the trusted public
    key."
    return TOOL_ERROR, "Cosign could not complete verification; failing closed."
```

The policy gate itself is deliberately tiny, which makes the fail-closed property easy to review:

Listing 5.9: *trustdeploy/policy.py: decide()*

```
def decide(result):
    # Fail closed: only an explicit, successful verification is accepted.
    return ACCEPT if result.verified is True else REJECT
```

The policy is data, not code, so that the trusted keys and allowed registries can be changed through review without touching the gate:

Listing 5.10: *trusted/policy.json*

```
{
  "policy_name": "trustdeploy-default",
  "description": "Images must be referenced by digest after resolution and carry a valid Cosign
    signature from the TrustDeploy release key.",
  "trusted_public_keys": ["trusted/cosign.pub"],
  "allowed_registries": ["localhost:5001", "td-registry.local:5000"],
  "transparency_log": "disabled (offline key-based prototype)",
  "verification_timeout_seconds": 60
}
```

Figure 5.4 shows the signed image verified first directly with `cosign verify` and then through the TrustDeploy gate. The gate is given the mutable tag `:1.0.0`, resolves it to the approved digest, and accepts it (TC01).

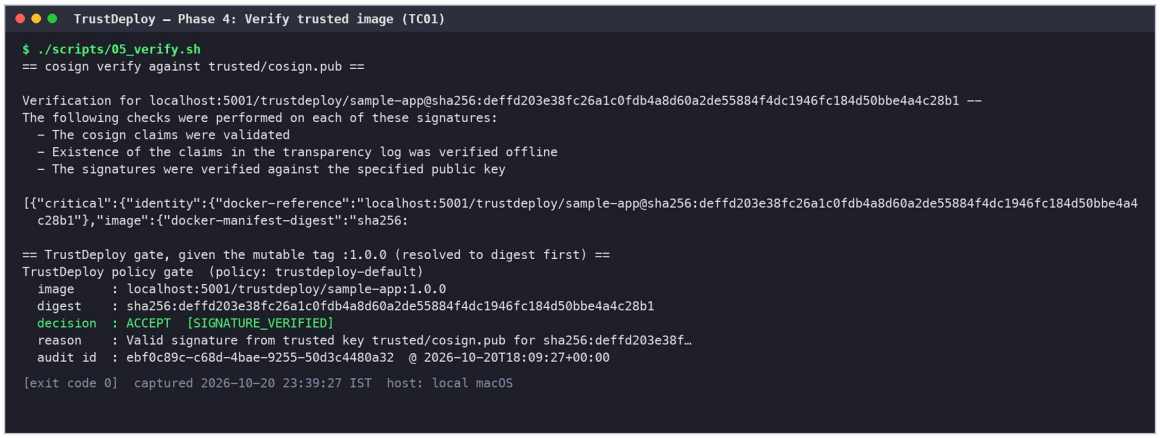


Figure 5.4: TC01: signed image verified with Cosign and accepted by the TrustDeploy gate *[05-tc01-verify-trusted.png]*

5.5 Phase 5: Attack Simulation and Untrusted Images

To test the negative paths, `scripts/06_prepare_untrusted.sh` creates one image per attack: an unsigned build (`1.1.0-unsigned`), a build signed with a separately generated attacker key (`1.0.1-attacker`), a build carrying a copy of the genuine signature (`1.0.2-forged`), and a tampered build (`tampered-build`) that is later moved onto the trusted tag. The attacker key has a different fingerprint from the trusted key (Figure 5.5).

The copied-signature attack is implemented in `scripts/forge_signature.py`. It talks to the registry API directly, takes the genuine signature manifest attached to the approved image, re-points its `subject` at the attacker's image and publishes it in the attacker image's referrers fallback index. The signed bundle layer is reused byte for byte, so the signature itself remains cryptographically valid:

Listing 5.11: *scripts/forge_signature.py (excerpt)*

```
    # 1. find the genuine signature manifest attached to A
    index = json.loads(req(base + src.replace(":", "-"))[0])
    sig_desc = index["manifests"][0]
    sig = json.loads(req(base + sig_desc["digest"], accept=MAN)[0])
    # 2. point its subject at B instead of A (the signed bundle layer is reused unchanged)
    body, headers = req(base + dst, method="HEAD")
    sig["subject"] = {"mediaType": headers["Content-Type"], "size": int(headers["Content-
    Length"]), "digest": dst}
    raw = json.dumps(sig).encode()
    new_digest = "sha256:" + hashlib.sha256(raw).hexdigest()
    req(base + new_digest, method="PUT", body=raw, ctype=MAN)
    # 3. publish it as B's referrer
    fake_index = {"schemaVersion": 2, "mediaType": IDX, "manifests": [dict(sig_desc,
    digest=new_digest, size=len(raw))]}
    req(base + dst.replace(":", "-"), method="PUT", body=json.dumps(fake_index).encode(),
    ctype=IDX)
```

Tag hijacking (TC04b) uses `scripts/retag.py`, which copies a manifest byte for byte under a new tag through the registry API, exactly as an attacker with push access could. The same script restores the tag to the approved digest after the test.

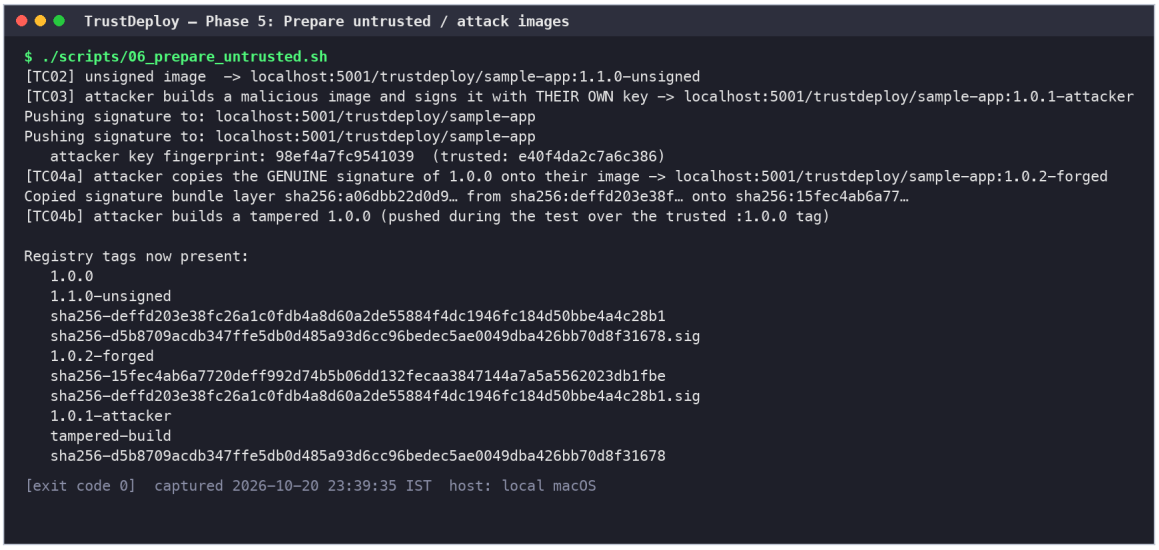


Figure 5.5: Preparation of the unsigned, attacker-signed, forged and tampered images *[06-prepare-untrusted-images.png]*

5.6 Phase 6: Policy Enforcement in the Deployment Workflow

The `deploy` command runs the gate and deploys only on ACCEPT. Crucially, it deploys `result.pinned_ref`, the digest that was verified, rather than the tag the user supplied. A rejected image ends the command with exit code 2 so that a CI pipeline step fails.

Listing 5.12: *trustdeploy/cli.py: deploy()*

```
def deploy_command(ref, target):
    """The deployment command for an already verified, digest-pinned reference."""
    if target == "docker":
        subprocess.run(["docker", "rm", "-f", "trustdeploy-app"], capture_output=True)
        return ["docker", "run", "-d", "--name", "trustdeploy-app", "-p", "18088:8080", ref]
    incluster = ref.replace("localhost:5001", "td-registry.local:5000")
    return ["kubectl", "-n", "trustdeploy", "set", "image", "deployment/sample-app",
    f"app={incluster}"]



def gate(image, action, label=None):
    decision, result, entry, policy = evaluate(image, action, label)
    colour = GREEN if decision == ACCEPT else RED
    print(f"{BOLD}TrustDeploy policy gate{END}  (policy: {policy['policy_name']})")
    print(f"  image     : {image}")
    print(f"  digest    : {result.digest or '-'}")
    print(f"  decision  : {colour}{BOLD}{decision}{END}  [{result.reason_code}]")
    print(f"  reason    : {result.message}")
    if result.tool_stderr and decision != ACCEPT:
        print(f"  evidence  : {result.tool_stderr.splitlines()[-1][:140]}")
    print(f"  audit id  : {entry['event_id']}  @ {entry['timestamp_utc']}")
    return decision, result



def deploy(image, target, label=None):
    decision, result = gate(image, "deploy", label)
    if decision != ACCEPT:
        print(f"{RED}{BOLD}DEPLOYMENT BLOCKED{END}: {image} did not satisfy the trust policy.
    Nothing was deployed.")
        return 2
    # always deploy the exact digest that was verified, never the tag
    cmd = deploy_command(result.pinned_ref, target)
    print(f"{GREEN}{BOLD}DEPLOYING{END} verified digest -> {target}: {' '.join(cmd)}")
    proc = subprocess.run(cmd, capture_output=True, text=True)
    print((proc.stdout + proc.stderr).strip())
    return 0 if proc.returncode == 0 else 1
```

Figure 5.6 shows the gate in a Docker deployment: the attacker-signed image is blocked with exit code 2 and no container is created; the approved image is then deployed by digest and answers on port 18088.

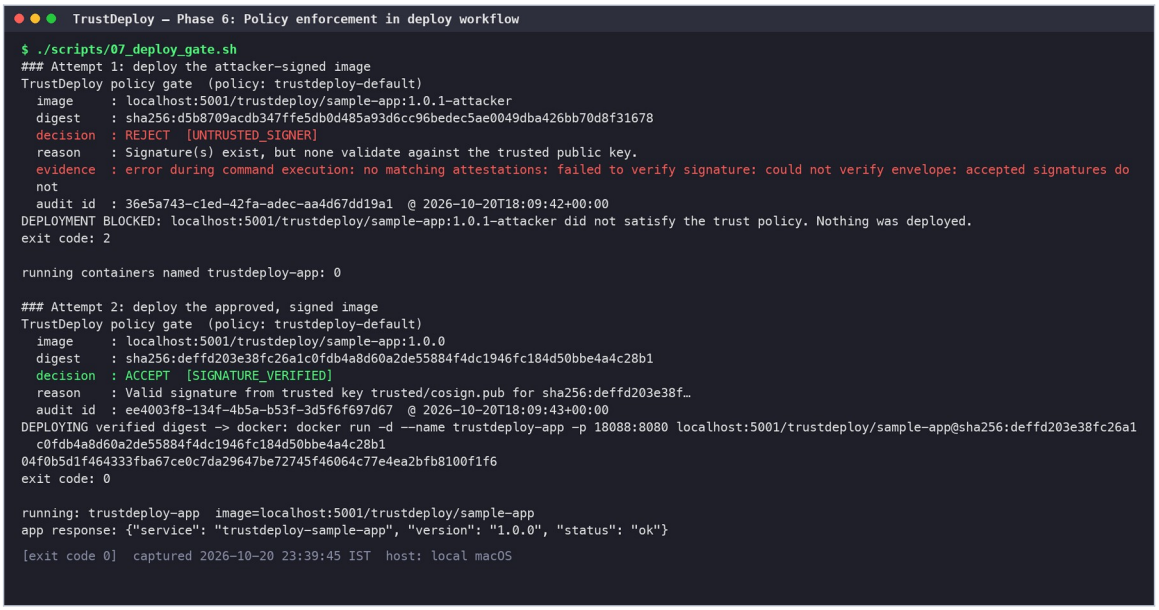


Figure 5.6: Deployment workflow: attacker image blocked (exit 2), approved image deployed by digest *[16-deploy-gate-docker.png]*

5.6.1 Kubernetes Admission Enforcement

To stop images that bypass the pipeline, the same key is enforced inside a local kind cluster by the Sigstore Policy Controller (Helm chart 0.10.8, application 0.13.1). The node's containerd is configured to read registry host configuration from `/etc/containerd/certs.d`, so it can pull from the plain-HTTP local registry. The `trustdeploy` namespace is labelled `policy.sigstore.dev/include=true` and the following `ClusterImagePolicy` requires every image from the TrustDeploy registry to be signed by the release key:

Listing 5.13: *k8s/cluster-image-policy.yaml*

```
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: trustdeploy-signed-images
spec:
  images:
    - glob: "td-registry.local:5000/**"
  authorities:
    - name: trustdeploy-release-key
      key:
        hashAlgorithm: sha256
        data: |
          -----BEGIN PUBLIC KEY-----
          MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEY2b6FmBx0DgxS6U62bBEmCVBe5CK
          AV4VHFUgHPELCYOUwm5WMSa4Y9tspN9Mj7+8kKT+Adohn6AeA8R/AHH3BA==
          -----END PUBLIC KEY-----
```

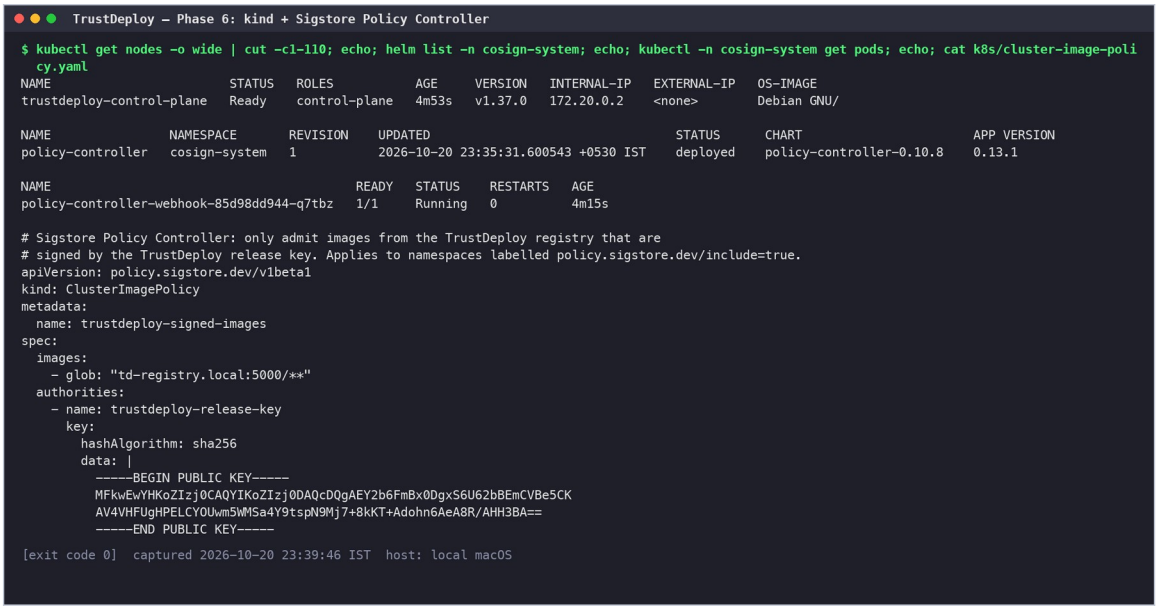


Figure 5.7: kind cluster, Policy Controller release and webhook pod, and the ClusterImagePolicy *[17-k8s-setup.png]*

Figure 5.8 shows admission enforcement in action. The Deployment that references the signed digest is admitted and its Pod reaches `Running`. A Pod using the attacker-signed image is denied by the `policy.sigstore.dev` webhook with "invalid signature when validating ASN.1 encoded signature", and a Pod using the unsigned image is denied with "no signatures found". Only the trusted Pod exists in the namespace afterwards.

```
TrustDeploy — Phase 6: Kubernetes admission enforcement

$ ./scripts/08_kubernetes.sh
Cluster: kind-trustdeploy   Policy Controller: Running
NAME                        AGE
trustdeploy-signed-images   3m12s
trustdeploy   Active   3m12s   kubernetes.io/metadata.name=trustdeploy,policy.sigstore.dev/include=true

### 1. Deploy the trusted, signed image (by digest)
deployment.apps/sample-app created
deployment "sample-app" successfully rolled out
NAME READY STATUS
sample-app-6d99dc7d4c-7qwbj 1/1 Running

### 2. Try to deploy the attacker-signed image
Error from server (BadRequest): admission webhook "policy.sigstore.dev" denied the request: validation failed: failed policy: trustdeploy-signed-
images: spec.containers[0].image
td-registry.local:5000/trustdeploy/sample-app@sha256:d5b8709acdb347ffe5db0d485a93d6cc96bedec5ae0049dba426bb70d8f31678 signature key validation fa
iled for authority trustdeploy-release-key for td-registry.local:5000/trustdeploy/sample-app@sha256:d5b8709acdb347ffe5db0d485a93d6cc96bedec5ae004
9dba426bb70d8f31678: no matching signatures: invalid signature when validating ASN.1 encoded signature

### 3. Try to deploy the unsigned image
Error from server (BadRequest): admission webhook "policy.sigstore.dev" denied the request: validation failed: failed policy: trustdeploy-signed-
images: spec.containers[0].image
td-registry.local:5000/trustdeploy/sample-app@sha256:b9e40137b00c862a6b41ee5395c8d7c627e3724015431781e0820f76db70e0e9 signature key validation fa
iled for authority trustdeploy-release-key for td-registry.local:5000/trustdeploy/sample-app@sha256:b9e40137b00c862a6b41ee5395c8d7c627e3724015431
781e0820f76db70e0e9: no signatures found

Pods in namespace trustdeploy:
  sample-app-6d99dc7d4c-7qwbj Running
[exit code 0]  captured 2026-10-20 23:39:50 IST  host: local macOS
```

Figure 5.8: Kubernetes admission: signed image admitted; attacker-signed and unsigned images denied *[18-k8s-admission.png]*

5.7 Phase 7: Audit Logging and Reporting

Every call to the gate, whether `verify` or `deploy`, records exactly one audit entry before any deployment action is taken:

Listing 5.14: *trustdeploy/audit.py: record()*

```
def record(result, decision, policy, key_fps, action, label=None):
    entry = {
        "event_id": str(uuid.uuid4()),
        "timestamp_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "action": action,
        "test_label": label,
        "image_ref": result.image_ref,
        "resolved_digest": result.digest,
        "verified_ref": result.pinned_ref,
        "decision": decision,
        "reason_code": result.reason_code,
        "reason": result.message,
        "policy": policy["policy_name"],
        "trusted_key_sha256": key_fps,
        "tool_exit_code": result.tool_exit_code,
        "tool_stderr_excerpt": result.tool_stderr,
        "duration_ms": result.duration_ms,
        "operator": getpass.getuser(),
    }
    LOG.parent.mkdir(exist_ok=True)
    with LOG.open("a") as f:
        f.write(json.dumps(entry) + "\n")
    return entry
```

`python3 -m trustdeploy.cli report` exports the log as `audit/report.json`, `audit/report.csv` and a static, self-contained `audit/report.html` dashboard with summary cards, a reason-code breakdown and the full event table including Cosign's error excerpt as evidence. The export and the dashboard are shown in Figures 6.9 and 6.10 as test case TC07.

CHAPTER 6
TESTING

6.1 Test Strategy

Testing was performed at two levels. **End-to-end tests** in `tests/cases.sh` run the real gate against the real registry and Cosign, and assert the expected decision from the process exit code (0 means ACCEPT, 2 means REJECT). **Unit tests** in `tests/test_gate.py` replace the Cosign call with mocks to exercise the fail-closed logic deterministically, without a registry. All test cases were executed in one clean run by `scripts/run_all.sh`; every figure in this chapter is a capture of that run.

Listing 6.1: *tests/cases.sh: the assertion helper*

```
expect() {  # expect <ACCEPT|REJECT> <label> <command...>
  local want=$1 label=$2; shift 2
  "$@"; local code=$?
  local got=REJECT; [ $code -eq 0 ] && got=ACCEPT
  if [ "$got" = "$want" ]; then echo "RESULT $label: PASS (expected $want, got $got)"; else echo
    "RESULT $label: FAIL (expected $want, got $got)"; return 1; fi
}
```

6.2 Test Case Summary

Table 6.1: End-to-end test cases and observed outcomes

ID	Scenario	Expected	Observed (reason code)	Status
TC01	Correctly signed image, requested by mutable tag `:1.0.0`	ACCEPT	ACCEPT (SIGNATURE_VERIFIED)	Pass
TC02	Unsigned image `1.1.0-unsigned`	REJECT	REJECT (NO_SIGNATURE)	Pass
TC03	Image signed with an attacker's key	REJECT	REJECT (UNTRUSTED_SIGNER)	Pass
TC04a	Genuine signature copied to another image	REJECT	REJECT (SIGNATURE_IMAGE_MISMATCH)	Pass
TC04b	Trusted tag `:1.0.0` overwritten with a tampered build	REJECT	REJECT (NO_SIGNATURE)	Pass
TC05a	Cosign binary missing, valid image, deploy	Blocked	Blocked (VERIFICATION_TOOL_ERROR)	Pass
TC05b	Registry stopped, valid image, deploy	Blocked	Blocked (IMAGE_RESOLUTION_FAILED)	Pass
TC05c	Image from `docker.io` (not in allow-list)	REJECT	REJECT (REGISTRY_NOT_ALLOWED)	Pass
TC06	Batch of four images	4 separate audit entries	4 entries: 1 ACCEPT, 3 REJECT	Pass

ID	Scenario	Expected	Observed (reason code)	Status
TC07	Audit report export	HTML, CSV and JSON written	All three written; dashboard rendered	Pass

6.3 Test Case Details

6.3.1 TC01: Correctly Signed Image

The gate is given `localhost:5001/trustdeploy/sample-app:1.0.0`, resolves it to the approved digest `sha256:deffd203e38f...` and verifies the signature with `trusted/cosign.pub`. Result: ACCEPT with SIGNATURE_VERIFIED (Figure 5.4).

6.3.2 TC02: Unsigned Image

Image `1.1.0-unsigned` was built and pushed but never signed. Cosign reports "no signatures found" and the gate rejects it with NO_SIGNATURE.

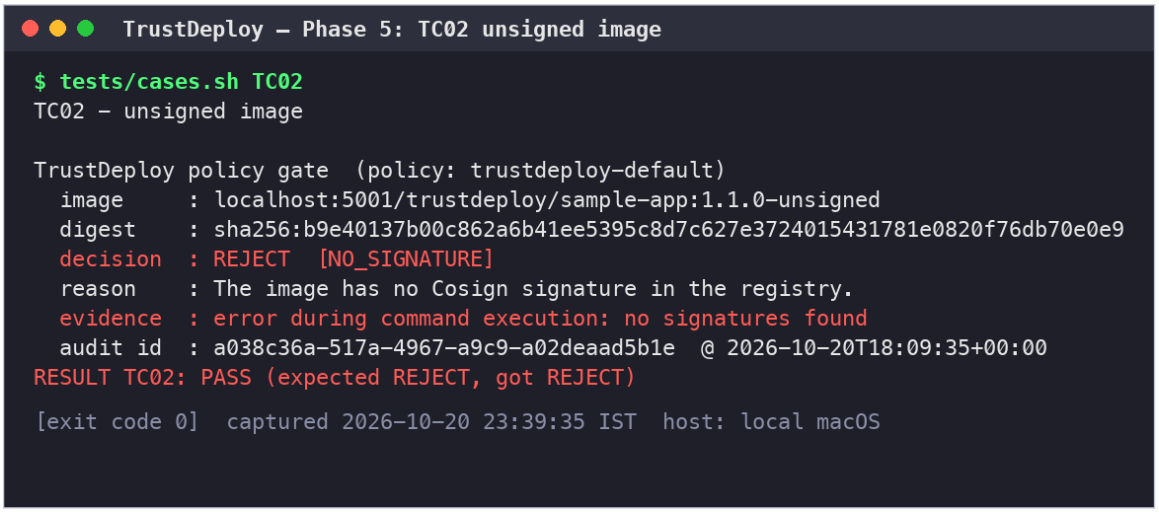


Figure 6.1: TC02: unsigned image rejected (NO_SIGNATURE) *[07-tc02-unsigned.png]*

6.3.3 TC03: Untrusted Signing Key

Image `1.0.1-attacker` *is* signed, but with the attacker's own key. Cosign finds the signature but it does not validate against the trusted public key, so the gate rejects it with UNTRUSTED_SIGNER. This demonstrates that "signed" is not the same as "trusted".

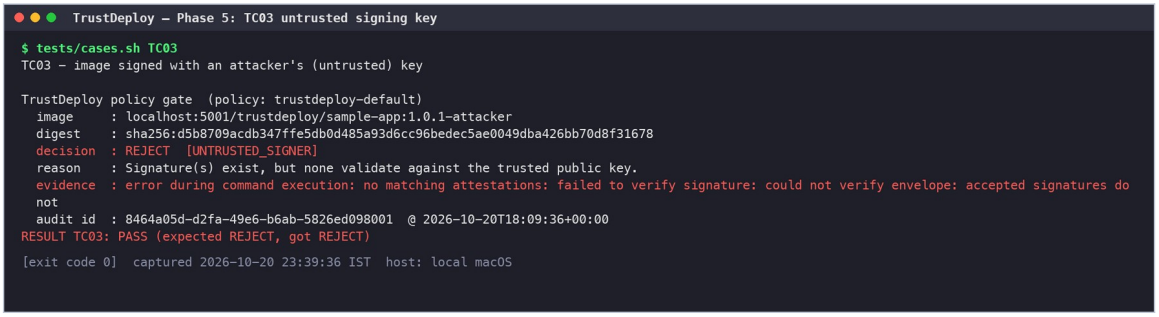


Figure 6.2: TC03: image signed with an attacker key rejected (UNTRUSTED_SIGNER) *[08-tc03-untrusted-key.png]*

6.3.4 TC04a: Genuine Signature Copied to Another Image

The genuine, valid signature of the approved image was attached to image `1.0.2-forged`. Because the signed statement names the approved image's digest, Cosign reports that the provided artefact digest does not match any digest in the statement and the gate rejects with SIGNATURE_IMAGE_MISMATCH. Signatures cannot be transplanted.



Figure 6.3: TC04a: copied genuine signature rejected (SIGNATURE_IMAGE_MISMATCH) *[09-tc04a-copied-signature.png]*

6.3.5 TC04b: Trusted Tag Overwritten

The tag `:1.0.0` was moved to the tampered build, simulating an attacker with registry push access. The gate resolves the tag, obtains the tampered image's different digest, finds no signature for that digest and rejects it. A tag-based deployment would have run the tampered image; the digest-based gate cannot be fooled. The tag is restored afterwards.

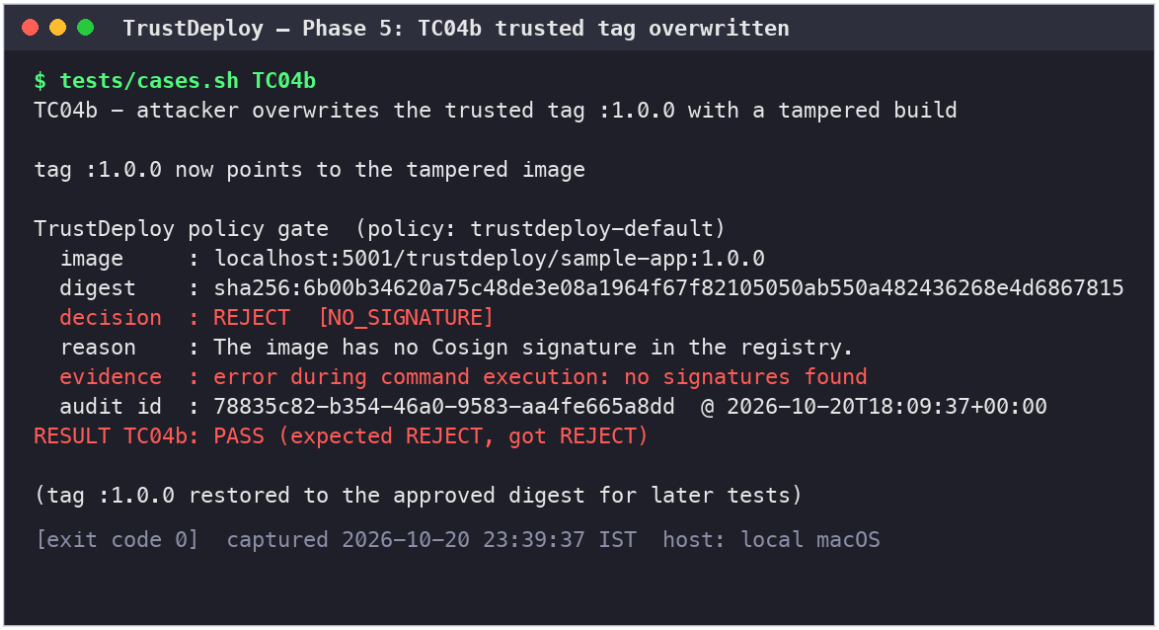


Figure 6.4: TC04b: trusted tag pointed at a tampered build, rejected (NO_SIGNATURE) *[10-tc04b-tag-hijack.png]*

6.3.6 TC05a: Verification Tool Unavailable

The gate is pointed at a non-existent Cosign binary (`TRUSTDEPLOY_COSIGN=/nonexistent/cosign`) while deploying the **valid** image. The verifier catches the `FileNotFoundError` and returns VERIFICATION_TOOL_ERROR; deployment is blocked. This proves that the gate fails closed even for a legitimate image.

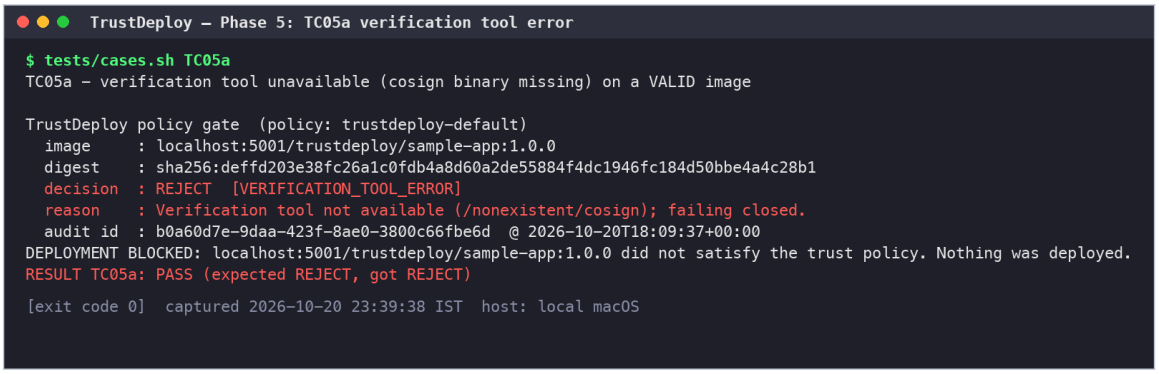


Figure 6.5: TC05a: Cosign missing, deployment of a valid image blocked (fail closed) *[11-tc05a-tool-missing.png]*

6.3.7 TC05b: Registry Unreachable

The registry container is stopped and the valid image is deployed. Digest resolution fails with a connection error, the gate returns IMAGE_RESOLUTION_FAILED and nothing is deployed. The registry is restarted after the test.

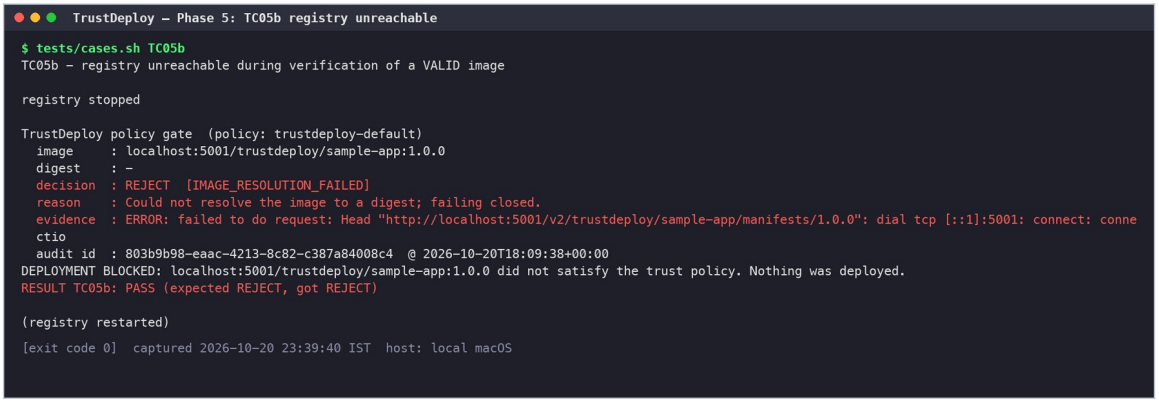


Figure 6.6: TC05b: registry unreachable, deployment blocked (IMAGE_RESOLUTION_FAILED) *[12-tc05b-registry-down.png]*

6.3.8 TC05c: Registry Not in Allow-list

`docker.io/library/nginx:latest` is rejected immediately with REGISTRY_NOT_ALLOWED. No network call is made, which is why the audit log records a verification time of 0 ms for this case.

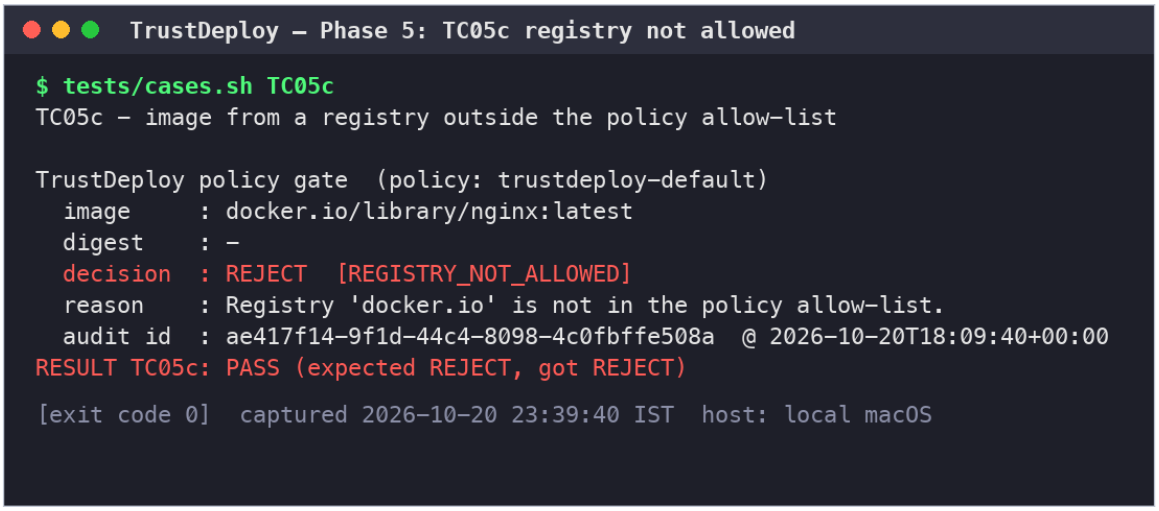


Figure 6.7: TC05c: image from a non-approved registry rejected (REGISTRY_NOT_ALLOWED) *[13-tc05c-registry-not-allowed.png]*

6.3.9 TC06: Multiple Images, Separate Audit Records

Four images are verified in one batch. The test counts the audit log lines before and after and asserts that exactly four entries were added, each with its own event ID, decision and reason code.

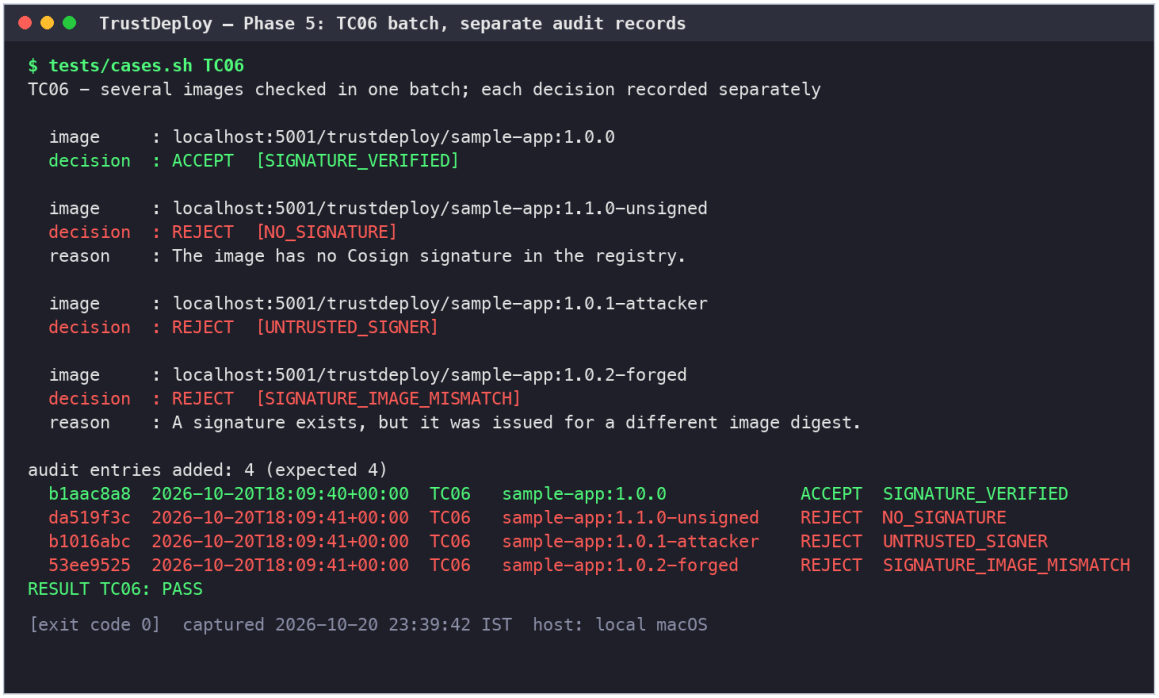


Figure 6.8: TC06: four images verified in one batch, four separate audit entries *[14-tc06-multiple-images.png]*

6.3.10 TC07: Audit Report Generation

The `report` command writes the HTML, CSV and JSON reports. Figure 6.9 shows the export with all audit entries of the run and a full sample record; Figure 6.10 shows the generated HTML dashboard.

```
TrustDeploy — Phase 7: TC07 audit report export

$ python3 -m trustdeploy.cli report && echo && echo "audit-log.jsonl entries: $(wc -l < audit/audit-log.jsonl)" && python3 scripts/audit_tail.py 30 &&
    echo && head -3 audit/report.csv && echo && echo "Sample audit record:" && tail -n 1 audit/audit-log.jsonl | python3 -m json.tool
Audit report written:
  /Users/amritnigam/TrustDeploy/audit/report.html
  /Users/amritnigam/TrustDeploy/audit/report.csv
  /Users/amritnigam/TrustDeploy/audit/report.json

audit-log.jsonl entries:       14
  ebf0c89c  2026-10-20T18:09:27+00:00  TC01   sample-app:1.0.0             ACCEPT  SIGNATURE_VERIFIED
  a038c36a  2026-10-20T18:09:35+00:00  TC02   sample-app:1.1.0-unsigned    REJECT  NO_SIGNATURE
  8464a05d  2026-10-20T18:09:36+00:00  TC03   sample-app:1.0.1-attacker    REJECT  UNTRUSTED_SIGNER
  d6f46d81  2026-10-20T18:09:36+00:00  TC04a  sample-app:1.0.2-forged      REJECT  SIGNATURE_IMAGE_MISMATCH
  78835c82  2026-10-20T18:09:37+00:00  TC04b  sample-app:1.0.0             REJECT  NO_SIGNATURE
  b0a60d7e  2026-10-20T18:09:37+00:00  TC05a  sample-app:1.0.0             REJECT  VERIFICATION_TOOL_ERROR
  803b9b98  2026-10-20T18:09:38+00:00  TC05b  sample-app:1.0.0             REJECT  IMAGE_RESOLUTION_FAILED
  ae417f14  2026-10-20T18:09:40+00:00  TC05c  nginx:latest                 REJECT  REGISTRY_NOT_ALLOWED
  b1aac8a8  2026-10-20T18:09:40+00:00  TC06   sample-app:1.0.0             ACCEPT  SIGNATURE_VERIFIED
  da519f3c  2026-10-20T18:09:41+00:00  TC06   sample-app:1.1.0-unsigned    REJECT  NO_SIGNATURE
  b1016abc  2026-10-20T18:09:41+00:00  TC06   sample-app:1.0.1-attacker    REJECT  UNTRUSTED_SIGNER
  53ee9525  2026-10-20T18:09:41+00:00  TC06   sample-app:1.0.2-forged      REJECT  SIGNATURE_IMAGE_MISMATCH
  36e5a743  2026-10-20T18:09:42+00:00  DEPLOY-ATTACKER sample-app:1.0.1-attacker    REJECT  UNTRUSTED_SIGNER
  ee4003f8  2026-10-20T18:09:43+00:00  DEPLOY-TRUSTED sample-app:1.0.0              ACCEPT  SIGNATURE_VERIFIED

timestamp_utc,test_label,action,image_ref,resolved_digest,decision,reason_code,reason
2026-10-20T18:09:27+00:00,TC01,verify,localhost:5001/trustdeploy/sample-app:1.0.0,sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c2
  8b1,ACCEPT,SIGNATURE_VERIFIED,Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
2026-10-20T18:09:35+00:00,TC02,verify,localhost:5001/trustdeploy/sample-app:1.1.0-unsigned,sha256:b9e40137b00c862a6b41ee5395c8d7c627e3724015431781e082
  0f76db70e0e9,REJECT,NO_SIGNATURE,The image has no Cosign signature in the registry.

Sample audit record:
{
    "event_id": "ee4003f8-134f-4b5a-b53f-3d5f6f697d67",
    "timestamp_utc": "2026-10-20T18:09:43+00:00",
    "action": "deploy",
    "test_label": "DEPLOY-TRUSTED",
    "image_ref": "localhost:5001/trustdeploy/sample-app:1.0.0",
    "resolved_digest": "sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1",
    "verified_ref": "localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1",
    "decision": "ACCEPT",
    "reason_code": "SIGNATURE_VERIFIED",
    "reason": "Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f\u2026",
    "policy": "trustdeploy-default",
    "trusted_key_sha256": [
        "e40f4da2c7a6c386"
    ],
    "tool_exit_code": 0,
    "tool_stderr_excerpt": "",
    "duration_ms": 291,
    "operator": "amritnigam"
}
[exit code 0]  captured 2026-10-20 23:39:51 IST  host: local macOS
```

Figure 6.9: TC07: audit report export (HTML, CSV, JSON) with the full audit trail *[19-tc07-audit-report.png]*

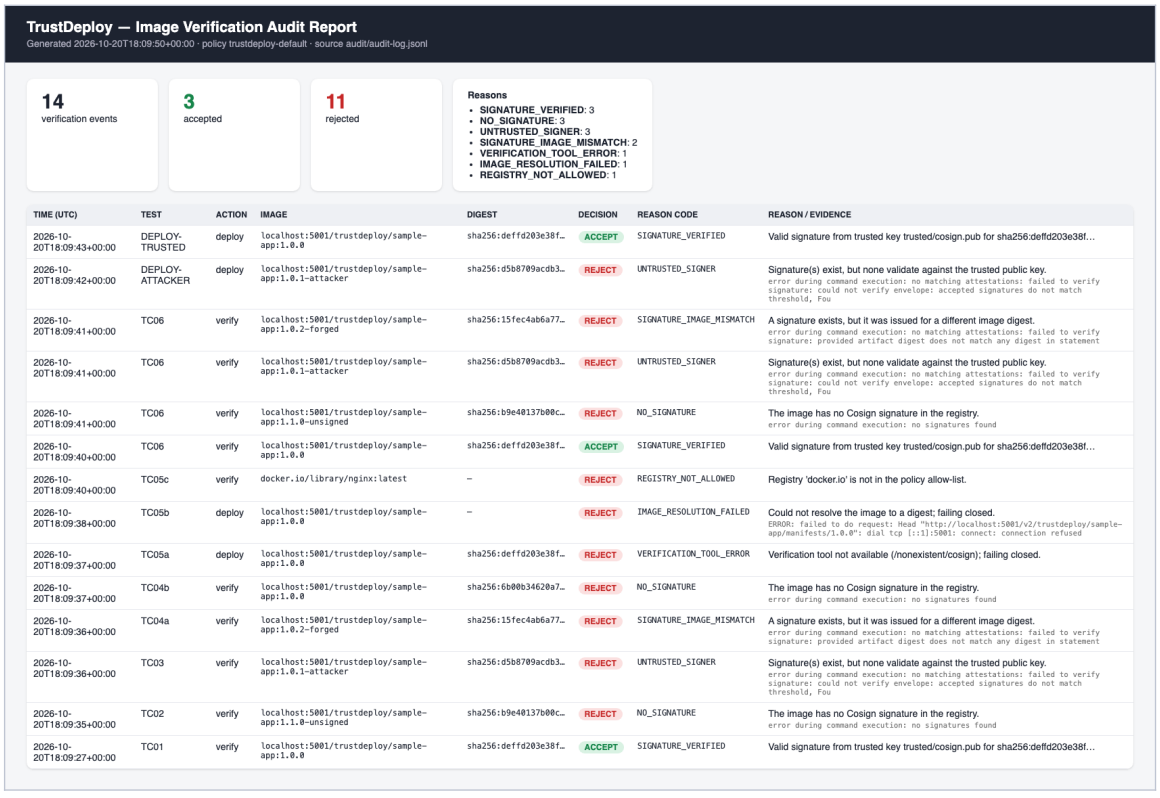


Figure 6.10: TC07: generated HTML audit dashboard *[20-audit-dashboard.png]*

6.4 Unit Tests

The eight unit tests in `tests/test_gate.py` target the decision logic with mocked Cosign results. All eight pass (Figure 6.11).

Table 6.2: Unit tests of the fail-closed gate

Test	What it checks
`test_valid_signature_accepted`	A successful verification whose payload names the digest is accepted.
`test_payload_for_other_digest_rejected_even_if_cosign_passes`	Even if Cosign exits 0, a payload for another digest is rejected (SIGNATURE_IMAGE_MISMATCH).
`test_failure_reasons_classified`	Cosign error messages map to NO_SIGNATURE, UNTRUSTED_SIGNER, SIGNATURE_IMAGE_MISMATCH and VERIFICATION_TOOL_ERROR.
`test_missing_tool_fails_closed`	A missing binary yields REJECT with VERIFICATION_TOOL_ERROR.
`test_timeout_fails_closed`	A verification timeout yields REJECT.
`test_unexpected_exception_fails_closed`	Malformed Cosign output yields REJECT with VERIFICATION_TOOL_ERROR.
`test_registry_not_allowed`	An image from docker.io is rejected with REGISTRY_NOT_ALLOWED.
`test_decide_requires_explicit_true`	A truthy but non-boolean `verified` value is still rejected.

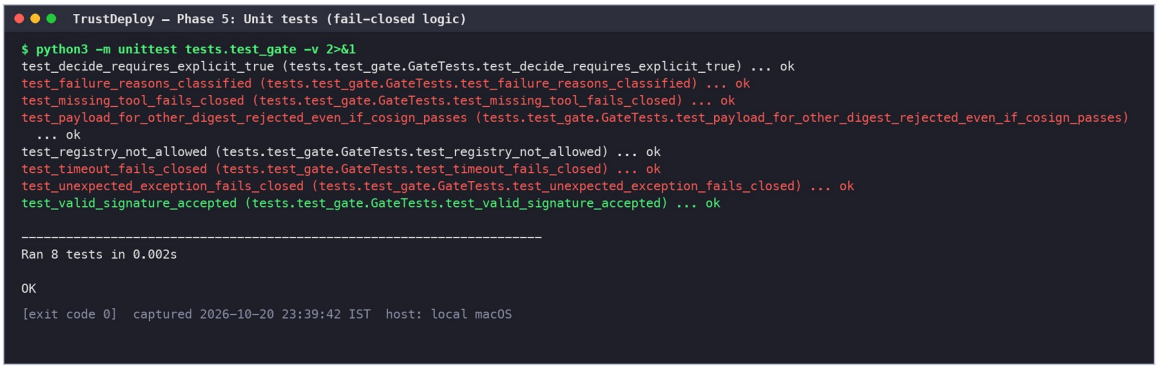


Figure 6.11: Unit tests: 8 of 8 passed *[15-unit-tests.png]*

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Summary of Results

The evaluation run recorded **15 audit events**: 4 ACCEPT and 11 REJECT (Figure 6.10). Every one of the ten end-to-end test cases and all eight unit tests produced the expected outcome. In addition, the Docker deployment workflow and the Kubernetes admission webhook both blocked the attacker-signed image while admitting the approved one. Table 7.1 shows the distribution of reason codes.

Table 7.1: Distribution of reason codes in the evaluation run

Reason code	Events
`SIGNATURE_VERIFIED`	4
`NO_SIGNATURE`	3
`UNTRUSTED_SIGNER`	3
`SIGNATURE_IMAGE_MISMATCH`	2
`VERIFICATION_TOOL_ERROR`	1
`IMAGE_RESOLUTION_FAILED`	1
`REGISTRY_NOT_ALLOWED`	1

7.2 Effectiveness Against the Threat Model

Table 7.2: Threat coverage

Threat	Control that stops it	Evidence
Unsigned image	Signature required by policy	TC02, Kubernetes step 3
Attacker-signed image	Only the trusted public key is accepted	TC03, deploy attempt 1, Kubernetes step 2
Copied signature	Signature bound to the digest; payload digest check	TC04a, unit test 2
Tag hijack	Resolve tag to digest, verify and deploy the digest	TC04b
Verifier or registry failure	Fail-closed gate	TC05a, TC05b, unit tests 4 to 6
Untrusted registry	Registry allow-list	TC05c
Missing accountability	Per-decision audit log and reports	TC06, TC07

7.3 Performance

Signature verification of an image in the local registry took between 248 ms and 594 ms per image in the evaluation run (mean 313 ms), including tag resolution and one `cosign verify` invocation. Failure paths that stop early are faster, and the allow-list check costs effectively nothing. This overhead is negligible compared with pulling and starting a container, so the gate can be placed on every deployment without a noticeable delay.

7.4 Discussion

Three observations stand out. First, the most dangerous attacks in the test set (TC03 and TC04a) both involve *valid signatures*; a check that merely asks "is this image signed?" would have accepted them. The security property comes from binding the signature to a specific key and a specific digest. Second, TC04b shows that the practice of deploying by tag is itself a vulnerability, and that the fix is cheap: resolve once, verify the digest, deploy the digest. Third, the failure-mode tests (TC05a and TC05b) are what distinguish a security control from a best-effort check. In both cases the image was legitimate, yet the gate correctly refused to deploy because it could not *prove* legitimacy.

The Kubernetes extension shows that the same trust root can be enforced at more than one layer. The CLI gate gives fast feedback in the pipeline and a detailed audit trail; the admission webhook ensures that a manual `kubectl run` cannot bypass the policy. One practical lesson was format compatibility: Cosign v3 writes signatures as Sigstore bundles attached through OCI referrers by default, while the Policy Controller version used reads the legacy `.sig` format, so the release step signs in both formats with the same key.

CHAPTER 8

SECURITY CONSIDERATIONS AND LIMITATIONS

- **A signature is not a vulnerability scan.** A valid signature proves that the release key holder approved this exact image; it says nothing about whether the image contains vulnerable packages or malicious code introduced before signing. Signing should follow, not replace, scanning and review in the build pipeline.
- **Key compromise defeats the scheme.** Anyone holding the private key and its password can sign arbitrary images. The prototype stores an encrypted key on a laptop; production use requires a KMS or HSM, restricted signing identities, key rotation and a revocation procedure (removing the key from the policy and re-signing).
- **Transparency log disabled.** Signing uses `--tlog-upload=false` and verification `--insecure-ignore-tlog=true` so the prototype works offline. Without Rekor there is no public, tamper-evident record of signing events, and misuse of a stolen key would be harder to detect.
- **Local registry over plain HTTP.** `registry:2` on `localhost:5001` has no TLS and no authentication. This is acceptable on the loopback interface for a demonstration, but registry traffic in production must use TLS and access control. Note that signatures still protect integrity even over an untrusted transport.
- **Trust in the gate host.** The gate trusts the `cosign` and `docker` binaries and the files in `trusted/`. An attacker who can modify these on the deployment host can bypass the gate; they should be integrity-protected and changes reviewed.
- **Audit log integrity.** The JSON-lines log is append-only by convention, not cryptographically. It should be shipped to a write-once or centrally managed log store.
- **Scope of the CLI gate.** The CLI only protects deployments that go through it. The Kubernetes admission webhook closes this gap for the cluster, but only for namespaces labelled for enforcement.
- **Classification relies on Cosign error text.** Reason codes are derived from Cosign's messages, which may change between versions. Unknown messages safely fall back to VERIFICATION_TOOL_ERROR (still a REJECT), so a wording change can only reduce diagnostic precision, never security.

CHAPTER 9

COST ANALYSIS

TrustDeploy was designed to be local-first. Every component is open source and runs on the developer's own machine, so the prototype incurred **zero cloud cost**. Table 9.1 compares this with what a comparable cloud-hosted registry would cost.

Table 9.1: Cost of the prototype compared with a managed cloud registry

Item	TrustDeploy (local)	Managed registry (AWS ECR, for comparison)
Registry storage	Free (`registry:2` container on local disk)	Private repositories billed per GB-month (USD 0.10 per GB-month in the pricing page example); 500 MB per month free for one year for new customers [5]
Data transfer	None (loopback and Docker network)	Data in free; same-region transfer to AWS compute free; transfer to the internet or other regions billed [5]
Signing and verification	Free (Cosign, open source)	Free tools; optional KMS keys billed separately
Kubernetes cluster	Free (kind on Docker)	Managed cluster and worker nodes billed per hour
Admission controller	Free (Sigstore Policy Controller)	Free (same software)
Total for this project	**USD 0**	Small for one image, but grows with images, versions and traffic

The images used in this project are small (about 88 MB each as reported by the local Docker image list), so even on a managed registry the storage cost would be marginal. The point of the comparison is that the security controls themselves (signing, verification, admission control) add no licence cost; the only costs in a production deployment are the hosting of the registry and cluster, and optionally a managed KMS for the key.

CHAPTER 10

FUTURE ENHANCEMENTS

- **Keyless signing with Sigstore** using OIDC identities from the CI system (for example, GitHub Actions), with verification of the certificate identity and issuer instead of a static key.
- **Transparency log** upload to Rekor (public or private instance) and mandatory inclusion-proof verification.
- **KMS or HSM-backed keys** (Cosign supports cloud KMS URIs) with rotation and documented revocation.
- **Attestations:** sign SLSA provenance, an SBOM and a vulnerability scan result as in-toto attestations, and extend the policy to require them (for example, no critical CVEs).
- **CI/CD integration:** run the gate as a required pipeline step and publish the HTML report as a build artefact.
- **Multiple signers and thresholds:** require signatures from both the build system and a human approver.
- **Centralised, tamper-evident audit storage** with alerting on UNTRUSTED_SIGNER and SIGNATURE_IMAGE_MISMATCH.
- **TLS and authentication for the registry**, and a single signature format once the admission controller supports Sigstore bundles via OCI referrers.

CHAPTER 11

CONCLUSION

TrustDeploy demonstrates that container image supply chain security based on digital signatures can be built and verified end to end with open-source tools, on a single laptop, at no cost. The prototype builds an image, signs it by digest with a protected Cosign key, and enforces a fail-closed policy that admits only images signed by the trusted key, both in a deployment workflow and at Kubernetes admission time.

The evaluation reproduced the attacks and failure modes that matter in practice. Unsigned images, images signed with an attacker's key, a genuine signature transplanted onto another image, a hijacked tag, a missing verification tool, an unreachable registry and a non-approved registry were all blocked, each with a specific reason code, while the legitimately signed image was accepted and deployed by digest. Every decision was recorded in an audit log and exported as human- and machine-readable reports.

The project also makes the limits explicit: a signature establishes origin and integrity, not safety; the scheme is only as strong as the protection of the signing key; and production use should add a transparency log, hardware-backed keys, TLS and attestations. Within those limits, signing by digest and verifying with a fail-closed gate is a small change to a deployment pipeline that removes a whole class of supply-chain attacks.

REFERENCES

[1] Sigstore, "Quickstart with Cosign," Sigstore Documentation. *Available:* https://docs.sigstore.dev/quickstart/quickstart-cosign/

[2] Sigstore, "Verifying Signatures," Sigstore Documentation (Cosign). *Available:* https://docs.sigstore.dev/cosign/verifying/verify/

[3] Sigstore, "Policy Controller Overview," Sigstore Documentation. *Available:* https://docs.sigstore.dev/policy-controller/overview/

[4] Sigstore, "Policy Controller Installation," Sigstore Documentation. *Available:* https://docs.sigstore.dev/policy-controller/installation/

[5] Amazon Web Services, "Amazon Elastic Container Registry Pricing." *Available:* https://aws.amazon.com/ecr/pricing/

All online references were accessed on 20 October 2026.

APPENDIX A: REPRODUCING THE DEMONSTRATION

All commands are run from the project root. The private key password is read from `~/.trustdeploy/keys/password`.

Listing 11.1: *Command reference*

```
./scripts/run_all.sh                       # full run: phases 1-7, all tests, all screenshots

python3 -m trustdeploy.cli verify <image> [--label TCxx]
python3 -m trustdeploy.cli deploy <image> [--target docker|kind]
python3 -m trustdeploy.cli report         # audit/report.{html,csv,json}

tests/cases.sh TC01 ... TC06               # individual end-to-end cases
python3 -m unittest tests.test_gate -v     # unit tests

# exit codes: 0 accepted/deployed, 2 rejected (deployment blocked), 1 usage error
```